

A MAJOR PROJECT REPORT

On

“AI PORTFOLIO MAKER”

Submitted to

**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY,
HYDERABAD**

In partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY

In

Computer Science and Engineering

Submitted by

KONDA MANISHA

[228P1A0522]

Under the guidance of

Mrs. JIVERIA JABEEN

Asst. Professor, Dept. Of CSE



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

AAR MAHAVEER ENGINEERING COLLEGE

(Affiliated to JNTU Hyderabad, Approved by AICTE)

Vyasapuri, Bandlaguda, Post: Keshavgi, Hyderabad-500005

2022-2026



AAR MAHAVEER ENGINEERING COLLEGE

Vyasapuri, Bandlaguda, Post: Keshavgiri, Hyderabad- 500005

(Approved by AICTE, Affiliated to JNTU, Hyderabad)

(Sponsored by Mahaveer Educational Society)

EAPCET/ECET Counselling Code: **AARM** JNTUH College Code: **8P**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the thesis “**AI Portfolio Maker**” being submitted by

KONDA MANISHA

[228P1A0522]

In partial fulfillment for the award of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING** to the **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD**. This is a record of bona-fide Project Work carried out by her under my guidance and supervision. The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mrs. Jiveria Jabeen

Internal guide

Asst. Professor

Dept of CSE

AARM

Mrs. P. Vidhya Prathima

Project Coordinator

Asst. Professor

Dept of CSE

AARM

Mrs. Madiha Samreen

Head of Department

Professor & Head

Dept of CSE

AARM

EXTERNAL EXAMINER

Dr. S. Sudhakara Reddy

Professor & Principal

AAR Mahaveer Engineering College

ACKNOWLEDGEMENT

I would like to express my deep appreciation and gratitude to **Mrs. JIVERIA JABEEN, Assistant Professor, Department of C.S.E, my Project Guide**, for her skillful guidance, constant supervision, timely suggestions, keen interest, and encouragement in completing my project within the stipulated time.

I wish to express my sincere gratitude to **Mrs. P. VIDHYA PRATHIMA, Project Coordinator**, for her valuable guidance, continuous support, and encouragement extended to me throughout the development of this project.

I express my profound sense of gratitude to **Mrs. MADIHA SAMREEN, Professor & Head of Department, C.S.E**, who has provided valuable suggestions, corrections, and necessary facilities to successfully complete this project.

I express my heartfelt thanks to **Dr. S. SUDHAKARA REDDY, Principal, AAR Mahaveer Engineering College**, for providing an excellent academic environment in the college.

I also wish to express my gratitude to the **Members of Staff** and all others who helped me in more than one way. I would also like to thank the **Lab Assistants and Technical Support Staff** for their support and assistance throughout my project.

KONDA MANISHA

[228P1A0522]

DECLARATION

I hereby declare that the results embodied in this project report entitled **“AI Portfolio Maker”** has been carried out by me during the academic year 2025-2026 as a partial fulfillment of the award of the **Bachelor of Technology in Computer Science and Engineering submitted to the Department of CSE, AAR MAHAVEER ENGINEERING COLLEGE**. I have not submitted this report to any other university or organization for award of any other degree.

KONDA MANISHA

[228P1A0522]

8074310161

manishakonda2@gmail.com

ABSTRACT

The **AI Portfolio Maker** is an intelligent web-based platform designed to simplify and automate the process of creating professional portfolio websites. It is developed using the **MERN stack** and **Tailwind CSS**, and integrates **Google Gemini API** to extract and structure key details such as skills, projects, and experience from user resumes (PDF/DOCX) or manually entered data. The system generates clean, responsive, and customizable portfolio templates without requiring technical knowledge.

The system addresses the need for quick and efficient portfolio creation by providing automated resume parsing, template generation, live preview, and downloadable standalone HTML files that can be hosted anywhere. Unlike traditional portfolio builders that rely on manual input and design, this platform uses **AI-driven processing** to improve content structuring and presentation.

This project serves as an effective solution for **students, fresh graduates, freelancers, and job seekers**, enabling them to create professional portfolios quickly and easily.

Keywords:

AI Portfolio Maker, Resume Extraction, MERN Stack, Gemini API, Template Rendering, PDF/DOCX Processing, Tailwind CSS, Automated Web Portfolio

TABLE OF CONTENTS

S.NO	CHAPTERS	PAGE NO.
1	INTRODUCTION	1-3
1.1	Overview	1
1.2	Aim	1
1.3	Objectives	1-2
1.4	Scope of the project	2
1.5	Motivation	2-3
1.6	Key Features of the System	3
2	SYSTEM ANALYSIS	4-7
2.1	Limitations of Existing System	4
2.2	Proposed System	4-5
2.3	Software Requirements	5-6
2.4	Hardware Requirements	6
2.5	Functional Requirements	6-7
2.6	Non-Functional Requirements	7
3	LITERATURE SURVEY	8-11
3.1	Introduction	8
3.2	Existing Portfolio Creation Tools	8-9
3.3	Limitations of Existing Systems	9
3.4	AI-Based Resume Parsing & Content Extraction	9-10
3.5	Technologies Used in This Project	10-11
3.6	Summary	11
4	SYSTEM TECHNOLOGIES & TOOLS	12-15
4.1	MERN Stack	12
4.2	TypeScript	13
4.3	Tailwind CSS	13
4.4	Google Gemini 1.5 Flash API	13
4.5	Resume Parsing	13-14
4.6	JWT Authentication	14
4.7	REST API Design	14-15

4.8	Responsive Web Design	15
5	DESIGN	16-24
5.1	Architecture	16-19
5.1.1	System Architecture	16
5.1.2	Software Architecture	18
5.2	UML Diagrams	19-24
5.2.1	Use Case Diagram	20
5.2.2	Class Diagram	21
5.2.3	Sequence Diagram	22
5.2.4	Activity Diagram	23
6	IMPLEMENTATION	25-35
6.1	Development Environment Setup	25
6.2	Backend Implementation	25-28
6.3	Gemini API Integration	28
6.4	Resume Parsing	28-29
6.5	Template System	29-30
6.6	Frontend Implementation	31-34
6.7	Deployment	35
6.8	Sample Output	35
7	TESTING	36-41
7.1	Testing Objectives	36
7.2	Unit Testing	36-37
7.3	Integration Testing	38
7.4	System Testing	38-39
7.5	Performance Testing	40
7.6	Security Testing	40
7.7	Usability Testing	41
7.8	Test Summary	41
8	RESULTS AND DISCUSSION	42-44
8.1	Overall System Outcome	42
8.2	Resume Parsing Results	42
8.3	Gemini API Extraction Quality	42-43

8.4	Template Output Quality	43
8.5	HTML Editor Performance	43
8.6	Credit System Behavior	43
8.7	Security Outcomes	43-44
8.8	Performance Outcomes	44
8.9	Usability Outcomes	44
8.10	Comparison with Project Objectives	44
9	CONCLUSION AND FUTURE ENHANCEMENTS	45-47
9.1	Conclusion	45
9.2	Key Achievements	45
9.3	Limitations	46
9.4	Future Enhancements	46-47
9.5	Learning Outcomes	47
9.6	Final Remarks	47
	REFERENCES	48

LIST OF FIGURES

S.NO	NAME	PAGE NO.
1.	System Architecture	16
2.	Software Architecture	18
3.	Use Case Diagram	20
4.	Class Diagram	21
5.	Sequence Diagram	22
6.	Activity Diagram	23

LIST OF TABLES

S.NO	NAME	PAGE NO.
1.	Hardware Requirements	6
2.	Portfolio Templates	30
3.	Unit Test Cases	36-37
4.	System Test Cases	39
5.	Performance Test Results	40
6.	Security Test Cases	40
7.	Test Summary	41

LIST OF SCREENSHOTS

S.NO	NAME	PAGE NO.
1.	Template Selection Screen	30
2.	Home Page UI	32
3.	Dashboard Page	32
4.	Generate Page	33
5.	Editor Page	34
6.	History Page	34
7.	Sample Portfolio Output	35

CHAPTER 1

INTRODUCTION

1.1 Overview

- A **portfolio website** is one of the most effective ways for students and professionals to present their skills, projects, and experience to potential employers. Despite its importance, building one from scratch requires knowledge of web design and a considerable amount of time, which many students do not have.
- The **AI Portfolio Maker** is a web application developed to address this challenge. The user uploads a resume in **PDF or DOCX** format, selects a preferred template, and the system automatically generates a complete portfolio website. The entire process takes under one minute and requires no technical knowledge from the user.
- The application is built using the **MERN stack** (MongoDB, Express.js, React, Node.js) with TypeScript on the frontend and Tailwind CSS for styling. It integrates with the **Google Gemini API** to read and extract content from the uploaded resume, which is then used to populate a pre-designed HTML template. The final output is a standalone HTML file the user can download and host for free on platforms such as GitHub Pages.

1.2 Aim

- The aim of this project is to develop an **AI-powered web application** that automatically converts a user's resume into a professional portfolio website. The system is designed to require minimal input from the user and produce an output that is clean, well-structured, and suitable for sharing with employers or academic institutions.
- The project also demonstrates the practical application of AI and modern web technologies in solving a real problem faced by students and fresh graduates.

1.3 Objectives

The specific objectives of the AI Portfolio Maker are listed below:

1. To accept resume files in **PDF and DOCX** formats and extract their text content accurately.
2. To use the **Google Gemini API** to convert extracted resume text into structured data containing name, contact details, skills, projects, experience, and education.
3. To fill pre-designed **HTML templates** with the extracted data and generate a complete portfolio page without any manual input.

4. To provide a **live preview** of the generated portfolio and a built-in HTML editor for user customization.
5. To allow users to download the final portfolio as a **self-contained HTML file** that can be hosted on any platform at no cost.
6. To implement **user authentication** so that each user maintains a personal account with a secure login.
7. To apply a **daily credit system** that limits each user to five portfolio generations per day.
8. To store all generated portfolios in a **database**, allowing users to access, edit, or delete them at any time.
9. To ensure the application is **secure, responsive, and accessible** across all devices and screen sizes.

1.4 Scope of the Project

The scope of this project covers the following areas:

- A responsive **frontend application** built with React, TypeScript, and Tailwind CSS.
- A **backend server** built with Node.js and Express, handling file uploads, AI integration, authentication, and database operations.
- Integration with the **Google Gemini API** for automated resume reading and structured data extraction.
- Ten hand-crafted **HTML portfolio templates** covering distinct visual styles.
- A **MongoDB database** for storing user accounts and all generated portfolio records.
- Security features including **JWT authentication**, bcrypt password hashing, Zod input validation, and API rate limiting.
- Support for both **PDF and DOCX** resume file formats.

The project does not cover enterprise features such as team accounts, custom domain management, or built-in analytics. These are noted as areas for future development.

1.5 Motivation

- Many students complete their undergraduate program with strong technical knowledge but without a professional online presence. A **portfolio website** allows recruiters and academic evaluators to assess a candidate's work beyond the content of a resume. However, building one manually requires time and web development skills that most students lack.

- Most existing portfolio creation tools require users to re-enter all their information manually, even though that information already exists in their resume. Professional-quality templates are typically restricted to **paid plans**, making them inaccessible for the majority of students.
- No currently available free tool uses AI to read an uploaded resume and automatically fill a portfolio. This gap provided the primary motivation for this project. By using AI to handle the extraction and structuring of resume content, the **AI Portfolio Maker** removes the biggest barrier to portfolio creation and makes the process accessible to all users, regardless of their technical background.

1.6 Key Features of the System

The AI Portfolio Maker is built around the following core features:

- **Resume Upload:** Supports PDF and DOCX file formats. Text is extracted automatically without any manual input.
- **AI-Powered Extraction:** The Google Gemini API reads the resume text and returns a structured JSON object containing all key profile details.
- **Template System:** Ten professionally designed HTML templates are available, covering dark, light, minimal, and modern visual styles.
- **Live Preview and Editor:** Users can preview the generated portfolio and edit the HTML directly in a built-in split-screen editor.
- **Download as HTML:** The final portfolio is delivered as a standalone HTML file with no external dependencies, ready to host on GitHub Pages or any static hosting platform.
- **Portfolio History:** All generated portfolios are saved to the user's account. Users can view, edit, re-download, or delete past portfolios at any time.
- **Daily Credit System:** Each user receives five generation credits per day. Credits are designed to reset every 24 hours.
- **Secure Authentication:** User accounts are protected with JWT-based login, bcrypt password hashing, and request rate limiting.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Limitations of Existing System

The following limitations were identified in existing portfolio creation tools and platforms, which directly motivated the development of this project.

- **Manual Design and Effort:** Traditional portfolio creation requires users to manually design, format, and write HTML/CSS, making it inaccessible for non-technical users.
- **No Resume-Based Automation:** No existing free tool accepts an uploaded resume and automatically generates a complete portfolio from it.
- **Paid Templates and Features:** Most platforms with professional-quality templates and download options are restricted to paid subscriptions.
- **No Downloadable Output:** Website builders host portfolios on their own servers. Users cannot get a standalone HTML file to host on platforms such as GitHub Pages.
- **Inconsistent Output Quality:** Without AI, the completeness and professionalism of the portfolio depends entirely on how much the user fills in manually.
- **Not Student-Friendly:** Most tools target professionals with web design experience and are not designed for students or fresh graduates with limited technical knowledge.

2.2 Proposed System

The AI Portfolio Maker is a web-based application that automates the entire process of portfolio creation from a single resume upload.

- The user uploads a resume in PDF or DOCX format, selects a template, and the system generates a complete portfolio website automatically.
- Google Gemini API reads the resume text and extracts structured data including name, contact details, skills, projects, experience, and education.
- The extracted data is used to populate a pre-designed HTML template, producing a clean, responsive, and downloadable portfolio page.
- Users can preview the generated portfolio, edit the HTML using a built-in split-screen editor, and download the final file with one click.
- The system requires no technical knowledge, no manual re-entry of information, and no paid subscription.

2.2.1 Advantages of the Proposed System

- Fully automates portfolio generation using **AI-powered resume extraction**.
- Provides ten professionally designed, responsive templates with distinct visual styles.
- Supports both **PDF and DOCX** resume formats without any manual conversion.
- Built-in HTML editor allows real-time preview and customization before download.
- Output is a **self-contained HTML file** with no external dependencies, ready to host on GitHub Pages or any static platform.
- Secure user accounts with a daily credit system and full portfolio history.
- Works for both technical and non-technical users with no prior web development knowledge required.

2.3 Software Requirements

The following software tools and technologies are required to develop and run the AI Portfolio Maker.

2.3.1 Frontend

- **React 18 with TypeScript:** Component-based UI library used to build all frontend pages and interactions.
- **Tailwind CSS:** Utility-first CSS framework for styling all components with responsive, consistent layouts.
- **Vite:** Frontend build tool that provides fast development server and optimized production builds.
- **shadcn/ui and Framer Motion:** UI component library and animation library for a polished, professional interface.

2.3.2 Backend

- **Node.js (v18+):** Server-side JavaScript runtime environment for executing backend logic.
- **Express.js:** Web framework for building REST API routes, handling file uploads, and middleware management.
- **MongoDB Atlas:** Cloud-hosted NoSQL database for storing user accounts and generated portfolio records.
- **Mongoose:** ODM library for defining data schemas and interacting with MongoDB.

2.3.3 AI, Auth, and Security

- **Google Gemini 1.5 Flash API:** AI model for extracting and structuring resume content into JSON.
- **JWT (jsonwebtoken):** For stateless user authentication via signed tokens.

- **bcryptjs:** For secure password hashing with 10 salt rounds.
- **Helmet.js and express-rate-limit:** HTTP security headers and API rate limiting.
- **Zod:** Schema-based input validation on all incoming request bodies.

2.3.4 Parsing and Development Tools

- **pdf-parse:** Extracts plain text from uploaded PDF resume files.
- **mammoth:** Extracts plain text from uploaded DOCX resume files.
- **VS Code, Git, GitHub, Postman:** Development, version control, and API testing tools.

2.4 Hardware Requirements

The following minimum hardware configuration is required to develop and run the application.

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	500 MB free disk space
Display	Any modern display with browser support
Internet	Required for Gemini API and MongoDB Atlas

2.5 Functional Requirements

The system must perform the following core functions.

- Accept resume uploads in PDF and DOCX formats and extract their text content accurately.
- Send the extracted resume text to the Google Gemini API and receive structured JSON data containing all portfolio fields.
- Populate a selected HTML template with the extracted data and render a complete portfolio page.
- Provide a live preview of the generated portfolio and a built-in HTML editor for customization.
- Allow the user to download the final portfolio as a self-contained HTML file.
- Maintain a portfolio history per user account with options to view, edit, star, and delete past portfolios.

- Enforce a daily credit limit of five portfolio generations per user, resetting every 24 hours.
- Implement secure user registration, login, and account management using JWT authentication.

2.6 Non-Functional Requirements

The system must also satisfy the following quality and operational requirements.

- **Performance:** The portfolio generation process should complete under 15 seconds. API response and page load times should be minimal.
- **Security:** All passwords must be hashed. JWT tokens must expire after 7 days. File uploads must be restricted to PDF and DOCX only.
- **Scalability:** The MERN-based architecture must allow future addition of new templates, features, and user tiers without major restructuring.
- **Usability:** The interface must be simple enough for users with no technical background to generate a portfolio without assistance.
- **Compatibility:** The application must work correctly on all modern browsers and screen sizes, including mobile devices.
- **Maintainability:** Code must be modular and well-structured, separating concerns across controllers, services, routes, and models.

CHAPTER 3

LITERATURE SURVEY

3.1 Introduction

- A literature survey examines existing work, tools, and research related to the project. It helps identify what has been done in the field, what problems exist in current systems, and how the proposed system addresses those gaps.
- For the AI Portfolio Maker, this survey covers existing portfolio creation tools, their limitations, approaches to AI-based resume parsing, and the technologies used. This study guided the design decisions made during development.

3.2 Existing Portfolio Creation Tools

Existing tools fall into three main categories: general website builders, developer-focused platforms, and resume hosting services.

3.2.1 General Website Builders

- Wix, Squarespace, and Weebly provide drag-and-drop interfaces for building websites without code.
- **All content must be entered manually.** There is no option to upload a resume and auto-fill the portfolio.
- Advanced templates and custom domains are locked behind paid plans.
- The output is hosted on the platform's servers and cannot be downloaded as a standalone file.

3.2.2 Developer-Focused Platforms

- GitHub Pages, Netlify, and Vercel allow free hosting of static HTML websites built by developers.
- **Users must write their own HTML and CSS** and manage repository deployments.
- These platforms require web development knowledge and are not suitable for non-technical users.
- There is no automation for resume content extraction or portfolio template generation.

3.2.3 Resume Hosting Services

- VisualCV, Resume.io, and Kickresume allow users to create and share resumes through a public link.
- **Content is shown in a fixed resume-style layout**, not a customizable portfolio format.

- Standalone HTML download is not supported. Users cannot host the output independently.
- Most advanced features and templates require a paid subscription.

3.3 Limitations of Existing Systems

The following key limitations were identified from the review of existing tools, and directly motivated the development of the AI Portfolio Maker.

- **No AI-Based Automation:** No existing free tool uses AI to read an uploaded resume and extract its content automatically.
- **Manual Data Entry:** All tools require users to re-type their details even though the information already exists in their resume.
- **Paid Features:** High-quality templates and download options are restricted to paid plans, making them inaccessible to most students.
- **No Downloadable HTML Output:** Most builders host portfolios on their own servers. Users cannot get a standalone file to host on GitHub Pages or similar platforms.
- **No Resume Upload Support:** No existing free tool accepts a PDF or DOCX resume as a starting point for portfolio generation.
- **Not Designed for Students:** Most tools target professionals with web design experience. Students without technical knowledge struggle to navigate and use them effectively.

3.4 AI-Based Resume Parsing and Content Extraction

Resume parsing extracts structured information from an uploaded resume document. Three main approaches were studied and compared for use in this project.

3.4.1 Rule-Based Methods

- Uses predefined rules and patterns (e.g., phone number formats, section keywords) to extract resume fields.
- Fails when resumes use non-standard headings or unusual formatting not covered by the rules.
- Rules must be manually updated for every new resume layout, making maintenance difficult.

3.4.2 Machine Learning-Based Methods

- Models such as Named Entity Recognition (NER) are trained on labeled resume datasets to identify and extract sections.

- **Requires a large, well-labeled training dataset** and significant computational resources.
- Common in large-scale ATS systems, but impractical for an academic project due to data and infrastructure needs.

3.4.3 Large Language Model-Based Methods (Adopted)

- LLMs such as Google Gemini are pre-trained on large text corpora and can read and extract resume content without task-specific training.
- **No training data or custom model development is needed.** The model understands natural language out of the box.
- Works accurately across a wide variety of resume formats, lengths, and styles.
- Returns structured JSON output that directly populates the HTML portfolio templates.
- Also enhances content for improved clarity and professional presentation.

3.5 Technologies Used in This Project

The following technologies were selected based on project requirements, free-tier availability, and compatibility with the overall system design.

3.5.1 MERN Stack

- **MongoDB:** NoSQL database for storing user accounts, portfolios, and metadata.
- **Express.js:** Backend framework handling API routing, file uploads, and middleware logic.
- **React (with TypeScript):** Frontend library for building a fast, component-based, and type-safe user interface.
- **Node.js:** Server-side runtime for handling concurrent requests efficiently.

3.5.2 Tailwind CSS

- A utility-first CSS framework used for styling all pages in the application.
- Speeds up development, ensures visual consistency, and produces responsive layouts without custom CSS files.

3.5.3 Google Gemini API

- Gemini 1.5 Flash is used to extract and structure resume content into JSON via a detailed prompt.
- Selected for its free-tier availability, fast response time, and accuracy across varied resume formats.
- Requires no custom model training or dataset preparation.

3.5.4 JWT Authentication and Security

- **JWT (JSON Web Tokens)** provide stateless authentication. The server issues a signed token on login; the client sends it with every request.
- Passwords are hashed using bcryptjs (10 salt rounds). Input is validated with Zod. Rate limiting is applied to all API routes.

3.5.5 Resume Parsing Libraries

- **pdf-parse:** Extracts plain text from uploaded PDF resumes for processing by the AI service.
- **mammoth:** Converts DOCX files into plain text before sending to the Gemini API.

3.6 Summary

The literature survey identified clear gaps in existing tools and confirmed the technology choices for this project. Key findings are listed below.

- Existing portfolio tools require significant manual effort and offer no resume-based automation.
- Most quality features are locked behind paid plans, making them inaccessible to students.
- **LLM-based parsing (Google Gemini)** is the most suitable approach for this project, offering high accuracy with no training data required.
- The MERN stack with React, TypeScript, and Tailwind CSS provides a well-documented and scalable foundation for the system.
- The overall architecture is practical, secure, and well-suited for both academic demonstration and real-world use.

CHAPTER 4

SYSTEM TECHNOLOGIES AND TOOLS

4.1 MERN Stack

The MERN stack is a set of four technologies used together to build full-stack JavaScript web applications. Each letter in MERN stands for one component of the stack.

4.1.1 MongoDB

- MongoDB is a document-oriented NoSQL database that stores data in JSON-like BSON format.
- It is used in this project to store user account records and all generated portfolio documents.
- MongoDB Atlas is the cloud-hosted version used for deployment, providing automatic scaling and backups.
- Mongoose ODM is used on top of MongoDB to define schemas and models for the User and Portfolio collections.

4.1.2 Express.js

- Express.js is a lightweight and flexible web framework for Node.js.
- It is used to define all backend API routes, apply middleware, and handle HTTP requests and responses.
- Middleware such as Helmet.js, rate limiters, and CORS configuration are mounted through the Express app.

4.1.3 React 18

- React is a JavaScript library developed by Meta for building component-based user interfaces.
- React 18 is used in this project with TypeScript for all frontend pages and interactive components.
- State management is handled using React hooks (useState, useEffect, useContext) and context providers for authentication and theme.

4.1.4 Node.js

- Node.js is a JavaScript runtime built on the V8 engine that allows running JavaScript on the server side.
- It powers the backend server and handles file uploads, API calls to Gemini, database operations, and authentication logic.
- Node.js version 18 or higher is required for this project.

4.2 TypeScript

TypeScript is a strongly-typed superset of JavaScript developed by Microsoft. It adds optional static types, interfaces, and compile-time checks to JavaScript code.

- TypeScript is used on the entire frontend of this project to improve code quality and reduce runtime errors.
- It provides better IDE support including auto-completion, type inference, and refactoring tools.
- TypeScript compiles to standard JavaScript before being bundled by Vite for the browser.

4.3 Tailwind CSS

Tailwind CSS is a utility-first CSS framework that provides a large set of pre-built CSS classes. Instead of writing custom CSS, developers apply utility classes directly in HTML or JSX.

- Tailwind is used to style every component and page in the frontend without any separate CSS files.
- It supports responsive breakpoints (sm, md, lg, xl) to make the layout adapt to different screen sizes.
- Dark mode is implemented using Tailwind's dark: variant prefix, switching classes based on the selected theme.
- Vite compiles and purges unused Tailwind classes during the production build, resulting in minimal CSS output.

4.4 Google Gemini 1.5 Flash API

Google Gemini 1.5 Flash is a large language model API provided by Google. It is capable of reading and understanding long documents and generating structured text output.

- The API is called from the Node.js backend after the resume text is extracted from the uploaded file.
- A carefully constructed prompt is sent to the API containing the full resume text and instructions to return a JSON object with specific fields.
- The API returns structured data including name, email, phone, summary, skills, projects, experience, and education.
- The free tier of Gemini 1.5 Flash provides sufficient quota for development and demonstration. A second API key can be configured as a fallback.
- Gemini Flash is chosen over other models because it supports long-context inputs, returns clean JSON, and has a free daily quota.

4.5 Resume Parsing

Resume parsing refers to the process of extracting plain text from uploaded resume files. The AI Portfolio Maker supports two file formats.

4.5.1 PDF Parsing

- The pdf-parse library is used to extract raw text from PDF resume files.
- It reads the binary PDF buffer and returns all text content as a plain string, which is then passed to the Gemini API.
- Only PDF and DOCX file types are accepted. Other file types are rejected at the backend validation layer.

4.5.2 DOCX Parsing

- The mammoth library is used to extract plain text from DOCX resume files uploaded by the user.
- mammoth converts the internal XML structure of a DOCX file into clean text, ignoring styles and formatting.
- The extracted text is then sent to the Gemini API as part of the portfolio generation request.

4.6 JWT Authentication

JSON Web Token (JWT) is an open standard for securely transmitting information between parties as a compact, signed token.

- When a user logs in, the backend generates a JWT signed with a secret key stored in the environment variables.
- The token is sent to the frontend and stored in memory or local state. It is included in the Authorization header for all protected API requests.
- The authMiddleware.js middleware on the backend verifies the token signature and extracts the user ID from the payload.
- Tokens expire after 7 days. Any request with an expired or invalid token is rejected with a 401 response.
- Passwords are hashed using bcryptjs with 10 salt rounds before storage. Plain text passwords are never stored in the database.

4.7 REST API Design

The backend of the AI Portfolio Maker follows REST (Representational State Transfer) architectural principles for all API endpoints.

- Each resource (auth, portfolio) has its own set of routes organized in separate route files.
- **HTTP methods are used correctly:** GET for retrieval, POST for creation, PUT for updates, and DELETE for removal.
- All request bodies are validated using Zod schemas before being processed by the controller.

- Error responses follow a consistent JSON format with a message field and appropriate HTTP status codes.
- All routes are prefixed with /api/ to separate the REST API from any static file serving.

4.8 Responsive Web Design

Responsive web design ensures that a web application works correctly and looks good across different screen sizes and device types.

- The frontend uses Tailwind CSS breakpoints to apply different layouts for mobile, tablet, and desktop screens.
- The application sidebar collapses on smaller screens and is replaced by a hamburger menu for navigation.
- All ten portfolio templates are designed to be fully responsive using CSS media queries and flexible grid layouts.
- The live preview panel in the editor and the portfolio history cards are both responsive to viewport width.

CHAPTER 5

DESIGN

5.1 Architecture

The architecture of the **AI Portfolio Maker** follows a layered and modular design that separates the user interface, backend logic, AI processing, data storage, and final output generation. The system ensures smooth interaction between components by using REST APIs, structured data exchange, and a clear flow of operations from resume upload to portfolio generation.

The overall architecture focuses on **simplicity, high performance, scalability, and accurate AI-driven extraction**, making the system efficient and suitable for real-time usage.

This section explains the complete architecture in two parts:

- **5.1.1 System Architecture**
- **5.1.2 Software Architecture**

5.1.1 System Architecture

The System Architecture is based on the block representation shown in the diagram. It describes how data flows between major components of the AI Portfolio Maker.

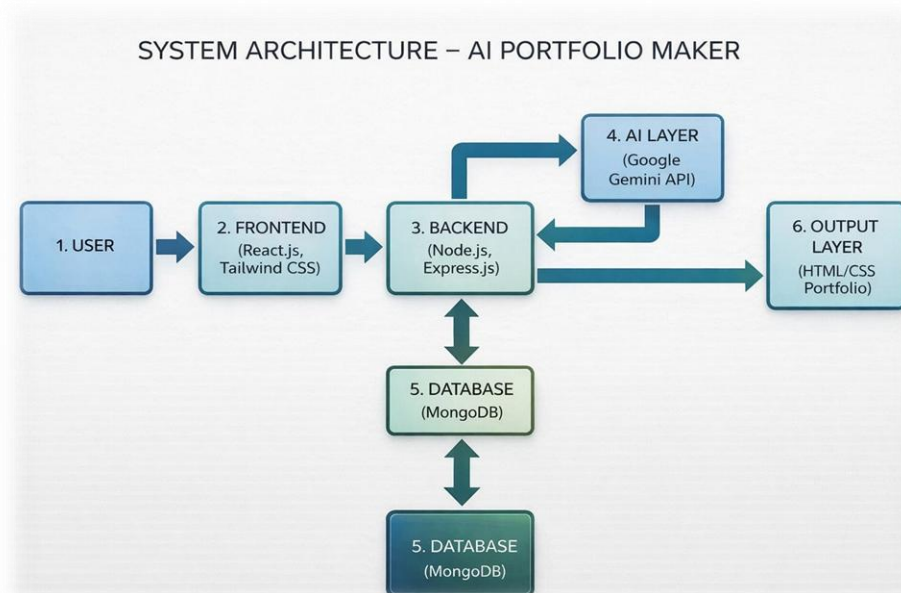


Figure 1: System Architecture Diagram

1. User

The user interacts with the system through a web browser. They upload their resume (PDF/DOCX), select a portfolio template, and preview or download the generated portfolio. All interactions begin and end at the user interface.

2. Frontend Layer (React.js, Tailwind CSS)

The **Frontend** is responsible for providing a clean, responsive, and user-friendly interface. Key functions include:

- Resume upload
- Template selection
- Viewing AI-processed content
- Live preview of the portfolio
- Triggering the download of the final HTML/CSS output

React.js handles dynamic rendering, while **Tailwind CSS** provides fast, consistent styling for templates and layouts.

3. Backend Layer (Node.js + Express.js)

The **Backend** acts as the core controller and communication hub of the entire system.

Its responsibilities include:

- Receiving uploaded resumes and template requests
- Sending requests to the **Gemini AI Layer** for extraction
- Handling API routing and validation
- Processing the structured data returned by AI
- Communicating with the database to store or fetch template details
- Returning final structured data to the frontend

The backend ensures that all processes run securely and efficiently.

4. AI Layer (Google Gemini API)

The **AI Layer** performs intelligent extraction and content enhancement.

Major tasks:

- Reading PDF/DOCX resumes
- Extracting skills, projects, experience, education, and summary
- Structuring the extracted information
- Enhancing text clarity and improving sentence quality

Gemini API transforms raw resume content into clean, organized data suitable for portfolio templates.

5. Database Layer (MongoDB)

MongoDB stores:

- Extracted user data (optional)
- Template definitions
- User details or saved portfolios (optional)

This layer allows the system to reuse templates, maintain consistency, and support future enhancements like user accounts.

6. Output Layer (HTML/CSS Portfolio)

After processing, the backend and frontend combine the extracted data with the selected template to generate:

- Final **HTML/CSS portfolio**
- A downloadable folder containing all files
- A preview displayed onscreen

This final output layer ensures the user receives a polished, deployable digital portfolio.

5.1.2 Software Architecture

The **Software Architecture** describes the internal structure and interaction between software components. It follows a **layered and modular approach**:

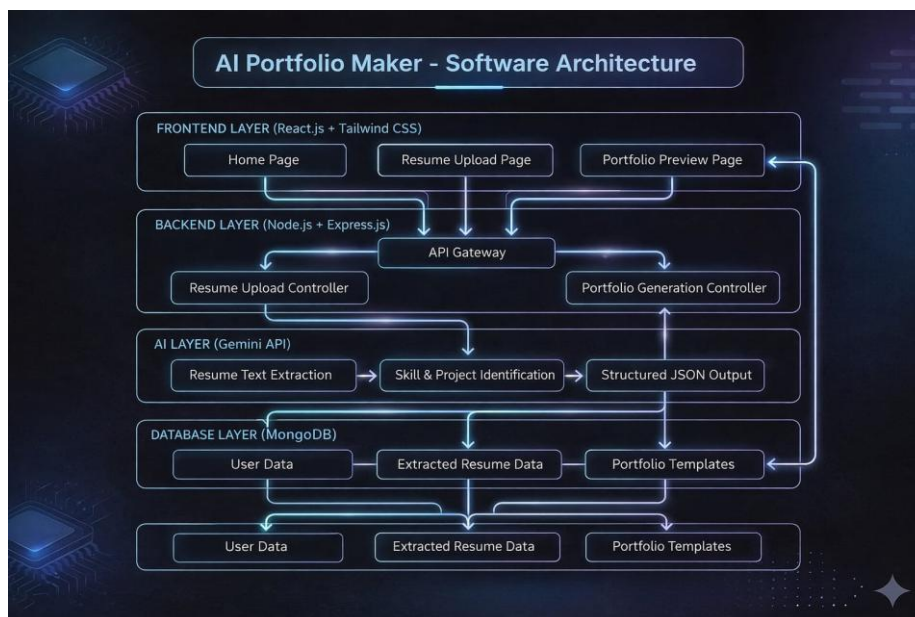


Figure 2: Software Architecture Diagram

1. Presentation Layer

- Built with **React.js**
- Handles all UI elements, resume upload, template preview, and user interactions

2. Application Layer

- Implemented using **Node.js & Express.js**
- Contains core logic, routing, file handling, and communication with AI and database layers

3. AI Service Layer

- Uses **Google Gemini API**
- Responsible for NLP, resume parsing, data structuring, and content enhancement

4. Data Layer

- Uses **MongoDB**
- Stores user data, templates, extracted content, and system metadata

5. Output Generation Layer

- Formats extracted data into ready-to-use HTML/CSS templates
- Generates downloadable or previewable portfolio files

This software architecture ensures a clean separation of responsibilities, easier maintenance, and smoother integration of new features.

5.2 UML Diagrams

UML diagrams are used to describe the functional and structural design of the **AI Portfolio Maker**. They help in understanding how the user interacts with the system, how data flows between components, and how the internal classes are organized. The following diagrams are used in this project:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram

5.2.1 Use Case Diagram

The **Use Case Diagram** shows the interaction between the **User** and the **AI Portfolio Maker** system. The user is represented as an actor, and the system is shown as a single main use case that provides multiple functionalities.

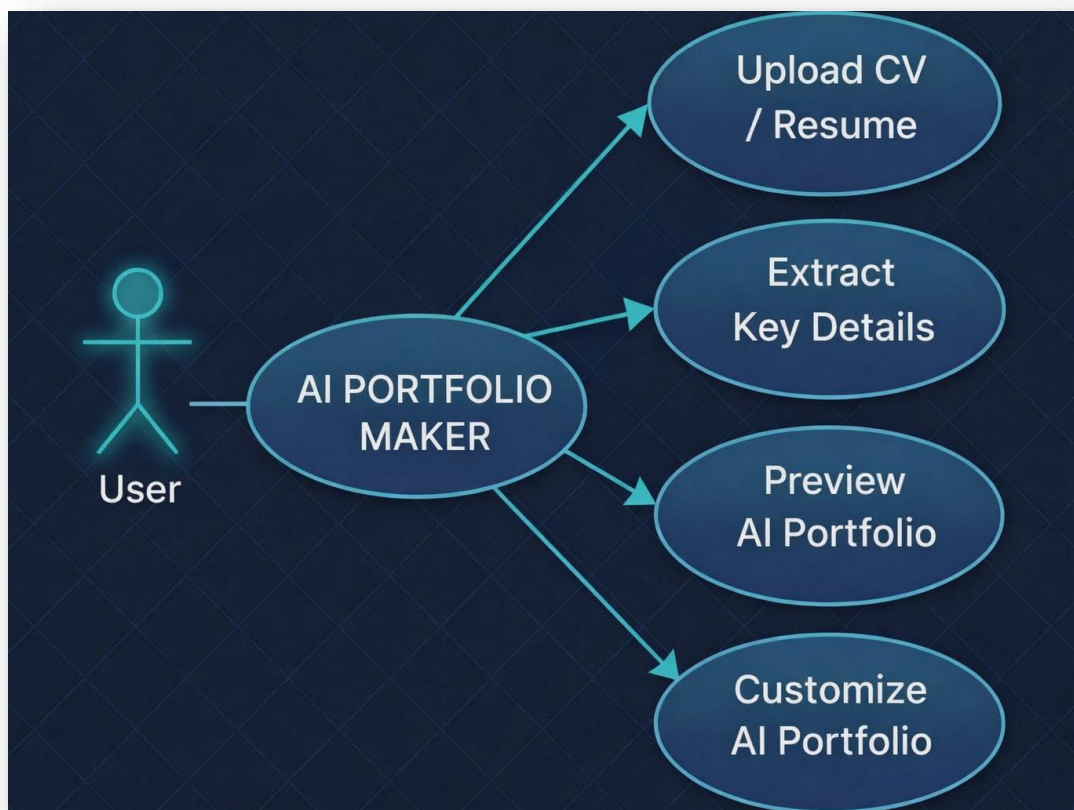


Figure 3: Use Case Diagram

Key use cases:

- **Upload Resume** – user uploads a PDF/DOCX resume.
- **Extract Details** – system extracts skills, projects, education, and experience using the AI layer.
- **Preview Portfolio** – system generates a preview of the portfolio using the selected template.
- **Customize Portfolio** – user can adjust content or layout before final download.

This diagram clearly shows that the entire process is driven by a single actor (user) and all major operations are performed by the AI Portfolio Maker system.

5.2.2 Class Diagram

The **Class Diagram** represents the internal structure of the system and its main classes. It describes how responsibilities are divided among components and how data is passed between them.

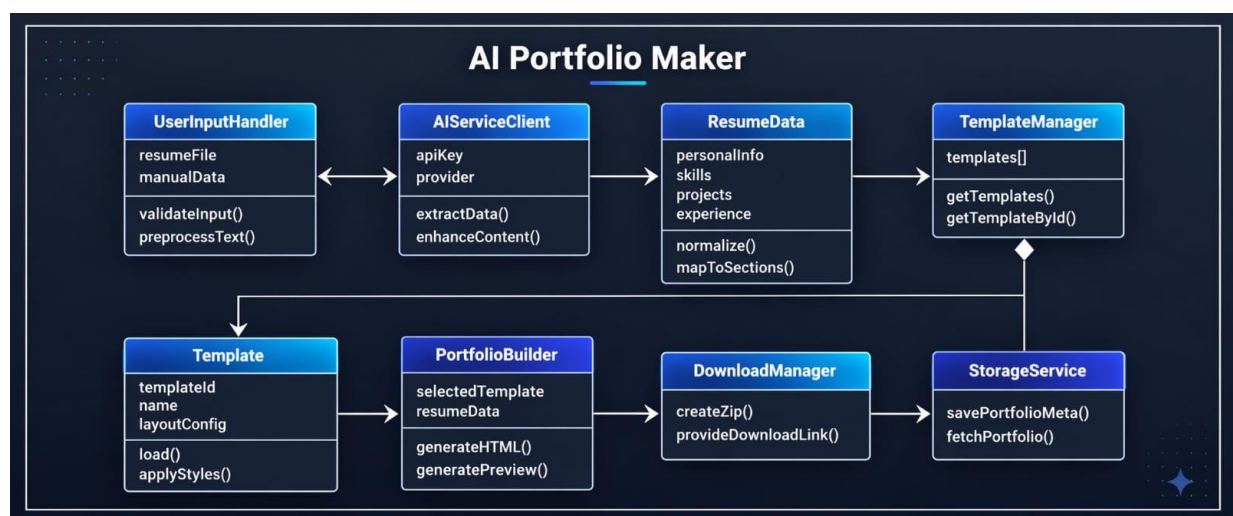


Figure 4: Class Diagram

Important classes:

- **UserInputHandler** – handles the uploaded resume file and any manually entered data; performs validation and pre-processing.
- **AIServiceClient** – communicates with the **Gemini API**; sends extracted text, receives structured data, and enhances content.
- **ResumeData** – stores structured information such as personalInfo, skills, projects, and experience; provides methods to normalize and map data to portfolio sections.
- **TemplateManager** – manages the list of available templates and provides methods like getTemplates() and getTemplateById().
- **Template** – represents a specific portfolio template with layout configuration and styling functions.
- **PortfolioBuilder** – combines the selected template and resume data to generateHTML() and generatePreview().

- **DownloadManager** – prepares downloadable ZIP files and provides download links.
- **StorageService** – optionally saves portfolio metadata and allows fetching stored portfolios.

Relationships between these classes show the overall flow from input handling → AI processing → data structuring → template loading → portfolio generation → download.

5.2.3 Sequence Diagram

The **Sequence Diagram** explains the step-by-step interaction between the main components over time. It focuses on the order of messages exchanged when a user generates a portfolio.

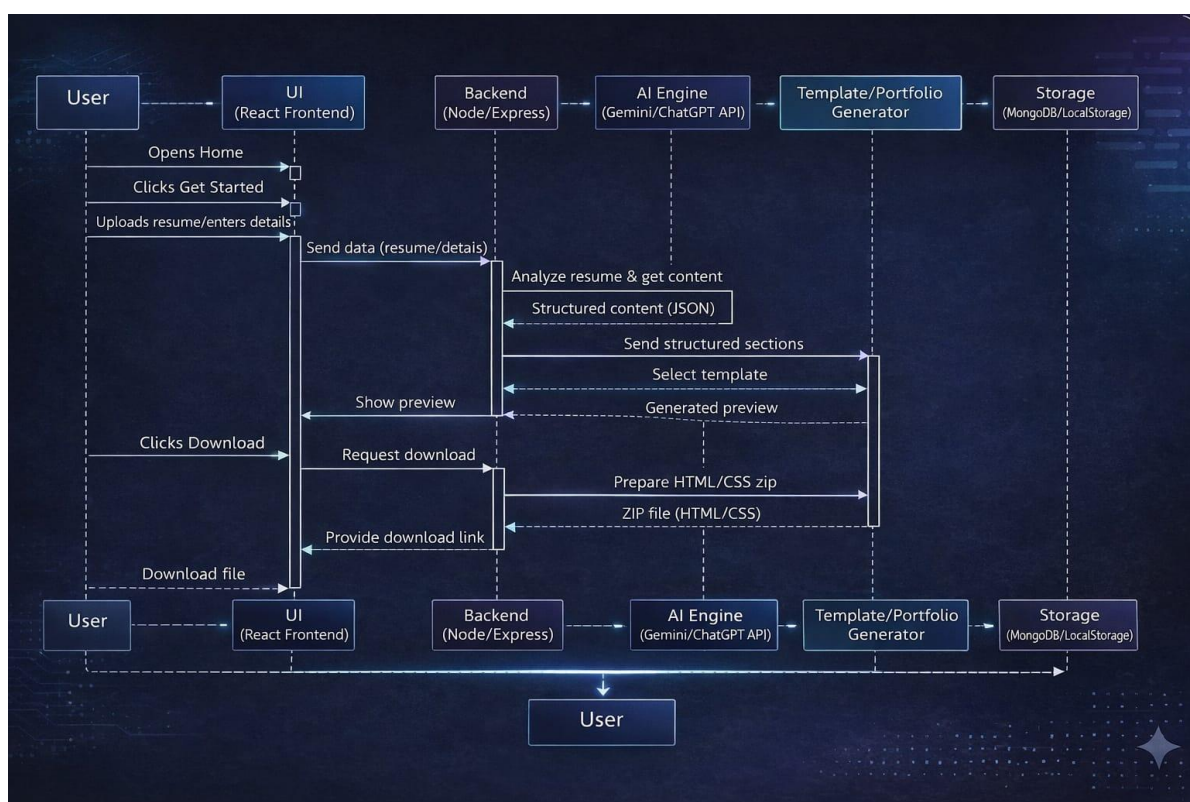


Figure 5: Sequence Diagram

Typical flow:

1. The user opens the home page and clicks **Get Started**.
2. The UI collects the resume/entered details and sends them to the backend.
3. The backend sends the data to the **Gemini API** for analysis and gets structured content in JSON format.

4. Structured sections are passed to the **Template/Portfolio Generator**, which selects a template and generates a preview.
5. The UI displays the preview to the user.
6. When the user clicks **Download**, the backend prepares an HTML/CSS ZIP file and stores any required metadata in storage.
7. A download link is returned to the UI and finally to the user.

5.2.4 Activity Diagram

The **Activity Diagram** represents the workflow of the AI Portfolio Maker as a series of actions and decisions.

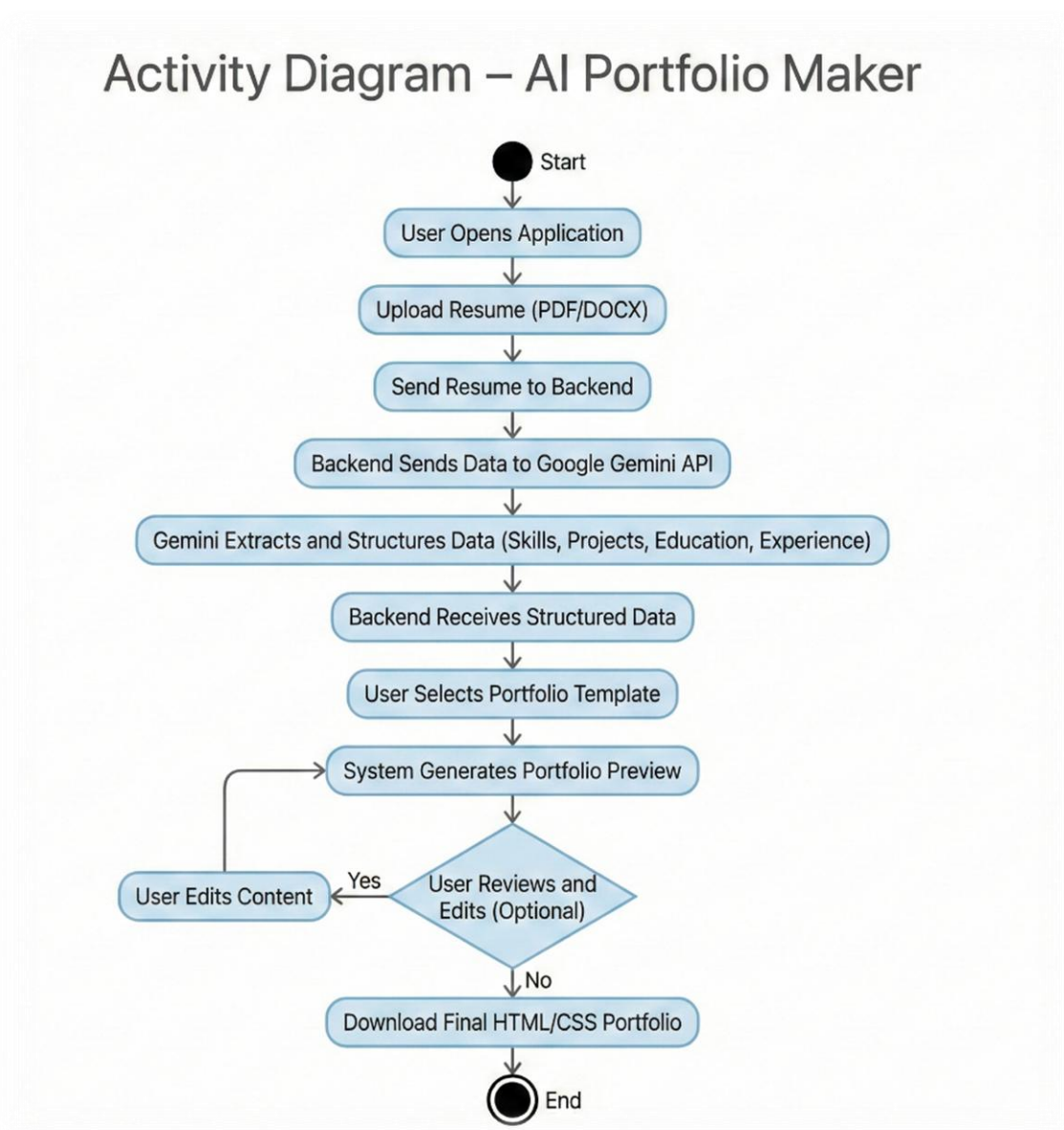


Figure 6: Activity Diagram

Main activities:

- Start → User Opens Application
- Upload Resume (PDF/DOCX)
- Send Resume to Backend
- Backend Sends Data to Gemini API
- Gemini Extracts and Structures Data
- Backend Receives Structured Data
- User Selects Portfolio Template
- System Generates Portfolio Preview
- Decision: User Reviews and Edits (Optional)
 - If Yes → User Edits Content → back to preview
 - If No → Download Final HTML/CSS Portfolio
- End

The activity diagram highlights both the straight-through flow and the optional editing loop, making the user journey and process logic easy to understand.

CHAPTER 6

IMPLEMENTATION

This chapter describes the implementation of the AI Portfolio Maker. The system is built as a backend REST API, a frontend React application, an AI integration layer, a resume parsing module, and a template engine. Each part communicates through well-defined interfaces.

6.1 Development Environment Setup

The following tools were configured before development began.

- **Node.js v18.x and npm v9.x:** JavaScript runtime and package manager used across both backend/ and frontend/ directories.
- **MongoDB Atlas:** Free-tier cloud database. The connection URI is stored in the backend .env file as MONGO_URI.
- **Google Gemini API Key:** Obtained from Google AI Studio and stored in GEMINI_API_KEY. A second key GEMINI_API_KEY_2 is supported as a fallback.
- **VS Code with ESLint and Prettier:** Code editor with linting and formatting extensions for consistent code style throughout the project.
- **Git:** Used for version control. The repository is hosted on GitHub.

The project uses a monorepo with two directories: backend/ running on port 5000, and frontend/ running on port 8080 via Vite. During development, all /api requests from the frontend are automatically proxied to the backend by the Vite configuration.

terminal

```
# Clone and install
git clone https://github.com/manoj-mukkamula/ai-portfolio-maker.git
cd ai-portfolio-maker

# Backend
cd backend && npm install && cp .env.example .env

# Frontend
cd ../frontend && npm install && npm run dev
```

Code 6.1: Project setup commands

6.2 Backend Implementation

The backend is a Node.js and Express.js application. The entry point is server.js, which starts the Express application defined in app.js. All files are organized under backend/ following a layered MVC structure.

6.2.1 Application Configuration (app.js)

app.js creates the Express application and registers all middleware and routes on every startup. Helmet.js sets HTTP security headers. CORS restricts origins to the configured frontend URL. The JSON body parser accepts up to 10 MB payloads. A rate limiter allows 100 requests per IP per 15 minutes. Auth routes are mounted at /api/auth and portfolio routes at /api/portfolio. A global error handler returns consistent JSON responses with appropriate HTTP status codes.

app.js

```
const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_URL }));
app.use(express.json({ limit: '10mb' }));
app.use(rateLimiter);
app.use('/api/auth', authRoutes);
app.use('/api/portfolio', portfolioRoutes);
app.use(errorHandler);
module.exports = app;
```

Code 6.2: Express application configuration

6.2.2 Data Models

The application uses two Mongoose models. The User schema stores name, unique email, hashed password, credit balance (default 5), and last credit reset timestamp. A pre-save hook hashes the password using bcryptjs with 10 salt rounds before every write, ensuring plain text passwords are never stored. The Portfolio schema stores an ObjectId reference to the owner user, the selected template name, the complete generated HTML string, and a starred flag.

models/

```
// User schema (models/User.js)
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  credits: { type: Number, default: 5 },
  lastCreditReset: { type: Date, default: Date.now },
}, { timestamps: true });

// Portfolio schema (models/Portfolio.js)
```

```
const portfolioSchema = new mongoose.Schema({
  owner:      { type: mongoose.Schema.Types.ObjectId, ref: 'User',
required: true },
  templateName: { type: String,   required: true },
  html:        { type: String,   required: true },
  starred:     { type: Boolean,   default: false },
}, { timestamps: true });
```

Code 6.3: User and Portfolio Mongoose schemas

6.2.3 Authentication Flow

The auth controller handles register, login, getMe, and deleteAccount. All request bodies are validated with Zod before any database operation. On register, the email is checked for duplicates. On login, bcryptjs.compare verifies the password; a 401 is returned on mismatch. Successful responses include a signed JWT that expires after 7 days. The authMiddleware extracts the Bearer token from the Authorization header, verifies it with jwt.verify, and attaches the decoded user to req.user. Missing or expired tokens return 401 immediately.

authMiddleware.js

```
// middleware/authMiddleware.js

const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Not authorized' });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch {
    res.status(401).json({ message: 'Token invalid or expired' });
  }
};
```

Code 6.4: JWT authentication middleware

6.2.4 Portfolio Controller (generatePortfolio)

This is the central function in the application. When a generation request arrives, it executes the following steps in order: the uploaded file or pasted text is parsed to extract plain text; the credit balance is checked and a 403 is returned if zero with no reset due; one credit is deducted and saved; the text is sent to the Gemini service which returns structured JSON; the JSON is

used to replace all {{PLACEHOLDER}} tokens in the selected HTML template; the completed HTML is saved as a new Portfolio document; and the portfolio ID and HTML are returned with a 201 status. Every read, update, and delete operation on a portfolio also verifies that the owner field matches req.user._id, returning 403 on mismatch.

6.3 Gemini API Integration

The Gemini API integration lives in services/geminiService.js. It accepts resume plain text and returns a structured JavaScript object. The prompt instructs the model to return only a valid JSON object with no extra text, no markdown, and no code fences. The required fields are: name, email, phone, location, summary, skills (array), projects (array of title, description, technologies), experience (array of company, role, duration, description), and education (array of institution, degree, year). If GEMINI_API_KEY_2 is defined, the service alternates between both keys on each call to extend the free-tier daily quota.

geminiService.js

```
// services/geminiService.js

const extractFromResume = async (resumeText) => {
  const key = getNextKey(); // alternates KEY and KEY_2
  const genAI = new GoogleGenerativeAI(key);
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
  const result = await model.generateContent(buildPrompt(resumeText));
  return JSON.parse(result.response.text());
};
```

Code 6.5: Gemini API call and JSON extraction

6.4 Resume Parsing

Resume parsing is handled in utils/resumeParser.js. The pdf-parse library reads the text layer of a PDF buffer and returns a plain string. The mammoth library converts the internal XML of a DOCX file into clean plain text, ignoring all formatting. If the user pastes text directly, it is returned as-is with no parsing step. Only PDF and DOCX MIME types are accepted; any other type throws an error that returns a 400 Bad Request response to the client.

resumeParser.js

```
// utils/resumeParser.js

const parsePDF = async (buf) => (await pdfParse(buf)).text;

const parseDOCX = async (buf) => (await mammoth.extractRawText({ buffer: buf })).value;

const parseResume = async (file, pastedText) => {
```

```

    if (pastedText) return pastedText;

    if (file.mimetype === 'application/pdf') return parsePDF(file.buffer);

    if (file.mimetype.includes('wordprocessingml')) return
    parseDOCX(file.buffer);

    throw new Error('Unsupported file type');
};

```

Code 6.6: Resume parsing for PDF, DOCX, and pasted text

6.5 Template System

The template system consists of ten standalone HTML files, each stored as a JavaScript template literal in `frontend/src/lib/templates.ts`. Every template embeds all CSS rules and font references in a single file with no external dependencies. The output can be opened in any browser or hosted anywhere without modification.

Each template uses `{{FIELD_NAME}}` placeholder tokens. After the Gemini service returns the structured data, the portfolio controller replaces every token using `String.replace` with a global regex. Complex sections such as skills, projects, experience, and education are first converted to HTML fragment strings by dedicated builder functions before insertion.

`portfolioController.js`

```

// portfolioController.js (simplified)

const populateTemplate = (templateHtml, data) => {
    let html = templateHtml;

    html = html.replace(/{{NAME}}/g,          data.name || '');
    html = html.replace(/{{SUMMARY}}/g,       data.summary || '');

    html = html.replace(/{{SKILLS_HTML}}/g,
    buildSkillsHtml(data.skills));

    html = html.replace(/{{PROJECTS_HTML}}/g,
    buildProjectsHtml(data.projects));

    html = html.replace(/{{EXPERIENCE_HTML}}/g,
    buildExperienceHtml(data.experience));

    html = html.replace(/{{EDUCATION_HTML}}/g,
    buildEducationHtml(data.education));

    return html;
};

```

Code 6.7: Template placeholder replacement

Template Name	Category	Visual Style
Vanta Pro	Premium	Adaptive dark/light
Aurora Luxury	Premium	Dark, Modern
Swiss Precision	Premium	Light, Minimal
Kinetic Magazine	Premium	Light, Modern
Glass Terminal	Standard	Dark, Minimal
Obsidian Code	Standard	Dark, Minimal
Aurora Studio	Standard	Dark, Modern
Deep Dark Minimal	Standard	Dark, Minimal
Clean Light Unique	Standard	Light, Minimal
Brutalist Grid	Standard	Light, Modern

Table 6.1: Ten portfolio templates with their categories and visual styles

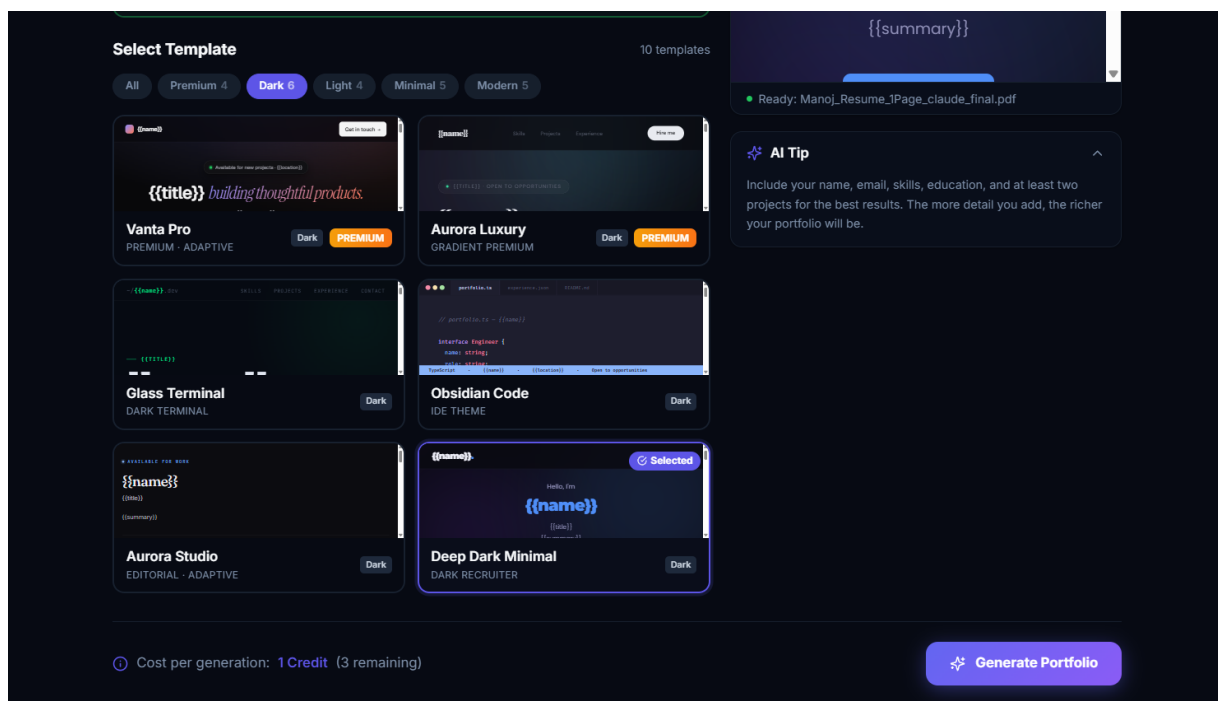


Figure 6.1: Template selection screen showing all ten templates with style tags and live preview buttons

6.6 Frontend Implementation

The frontend is built with React 18, TypeScript, and Vite. Pages are organized under `src/pages/` and all share the `AppLayout` component which renders the sidebar and top header. Tailwind CSS handles styling throughout.

6.6.1 Authentication and Theme

`AuthContext` holds the current user and JWT token. On load it calls `GET /api/auth/me` using any token in `localStorage`; if successful, the user is restored to a logged-in state. The `Axios` instance in `src/lib/api.ts` attaches the token to every outgoing request via a request interceptor. `ThemeContext` toggles a dark class on the document root so all Tailwind dark: variants apply globally. The chosen theme is persisted in `localStorage`.

6.6.2 AuthPage (Login and Registration)

- The `AuthPage` handles both user registration and login within a single page. A tab toggle switches between the two forms. The registration form collects name, email, and password. The login form collects email and password. Both forms perform client-side validation before sending the request. On successful login or registration, the JWT returned by the backend is stored in `AuthContext` and the user is redirected to the `DashboardPage`.
- If an error is returned, such as a duplicate email on registration or an incorrect password on login, an inline error message is displayed below the relevant field. The page is only accessible to unauthenticated users. If a logged-in user navigates to this route, they are redirected to the dashboard automatically.

6.6.3 AboutPage

The `AboutPage` describes the purpose and background of the AI Portfolio Maker. It explains what the application does, who it is built for, and how the underlying technology works at a high level. The page is accessible to both logged-in and guest users from the navigation header. It covers the problem the application solves, the technologies used to build it, and a brief note about the academic context of the project. No authentication is required to view this page.

6.6.4 ContactPage

The `ContactPage` provides the developer's contact information for users who want to report issues or ask questions. It displays the developer's name, email address, GitHub profile link, and LinkedIn profile link. The page is static and does not contain a form submission feature. It is accessible to all users regardless of login status.

6.6.5 HomePage

The `HomePage` is the public landing page of the application. It is the first screen a visitor sees before logging in. It presents the key features of the AI Portfolio Maker, a brief description of how the system works in three steps (upload, generate, download), and two call-to-action buttons: `Get Started`, which navigates to the registration page, and `View Templates`, which scrolls to the template preview section. The page is fully responsive and renders correctly on mobile and desktop viewports.

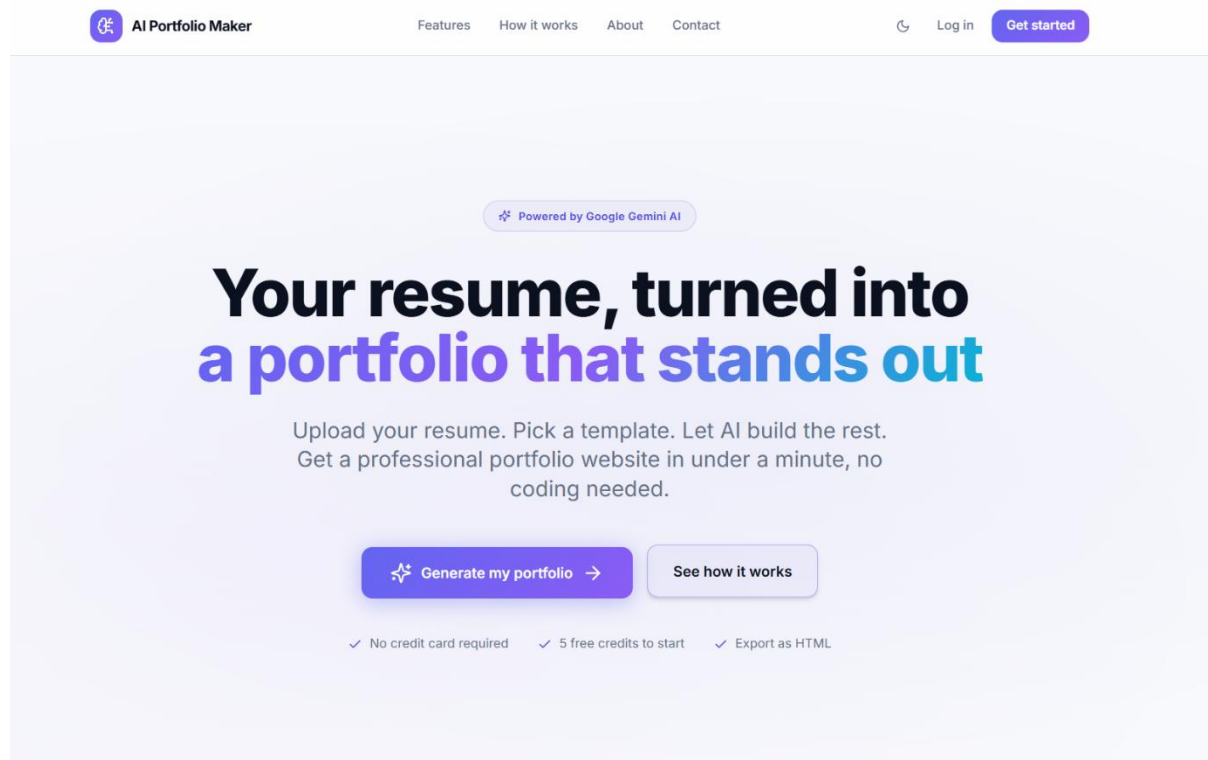


Figure 6.2: HomePage showing the hero section, three-step workflow, and call-to-action buttons

6.6.6 DashboardPage

The DashboardPage is the first screen a user sees after logging in. It displays the user's name, current credit balance, total portfolios generated, and the number of starred portfolios. Quick-action cards allow the user to navigate directly to GeneratePage or HistoryPage. A credit reset notice is shown when the daily limit has been reached, along with the time remaining until the balance resets. The dashboard adapts to both dark and light mode using the ThemeContext toggle.

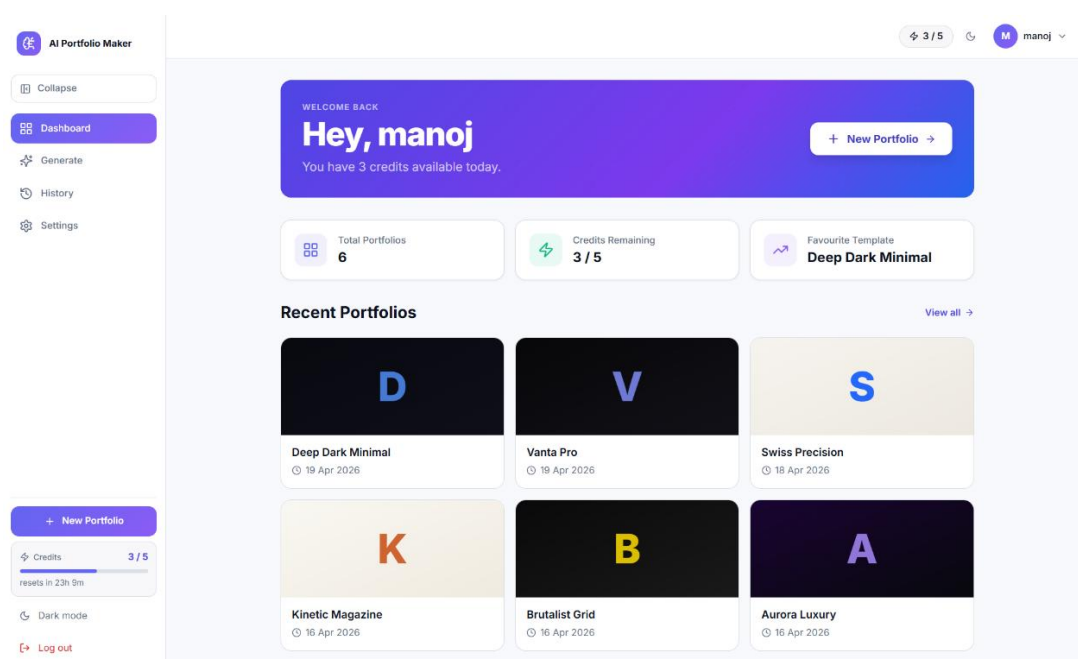


Figure 6.3: *DashboardPage* showing credit balance, portfolio count, and quick-action navigation cards

6.6.7 GeneratePage

This is the main page for creating a portfolio. The user uploads a PDF or DOCX file through a drag-and-drop zone, or pastes resume text into a textarea. Ten template cards are displayed from `templates.ts`, each showing a label, style tags, and a preview button that opens the blank template HTML in a modal iframe. Clicking Generate Portfolio sends a `FormData` request to `POST /api/portfolio/generate`. A loading spinner is shown during processing. On success the user is navigated to the `PreviewPage`.

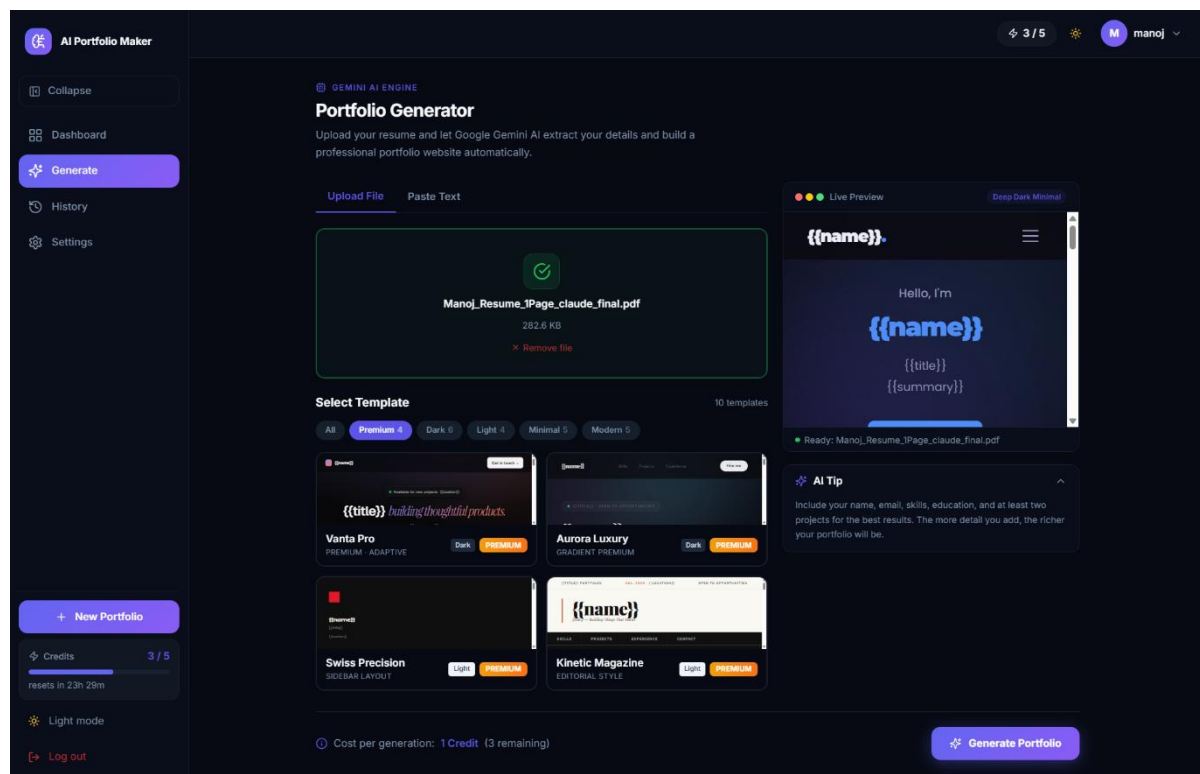


Figure 6.4: *GeneratePage* showing the resume upload zone, template selection cards, and the Generate button

6.6.8 PreviewPage and EditorPage

The `PreviewPage` renders the generated HTML inside a sandboxed iframe using a Blob URL so portfolio scripts cannot interfere with the application. A Download button triggers a client-side file download and an Open in Editor button navigates to the `EditorPage`.

The `EditorPage` shows a split-screen layout with a controlled HTML textarea on the left and a live preview iframe on the right. The iframe's `srcDoc` attribute is updated on every keystroke, producing real-time output. A Save button sends a PUT request to `/api/portfolio/:id` with the updated HTML.

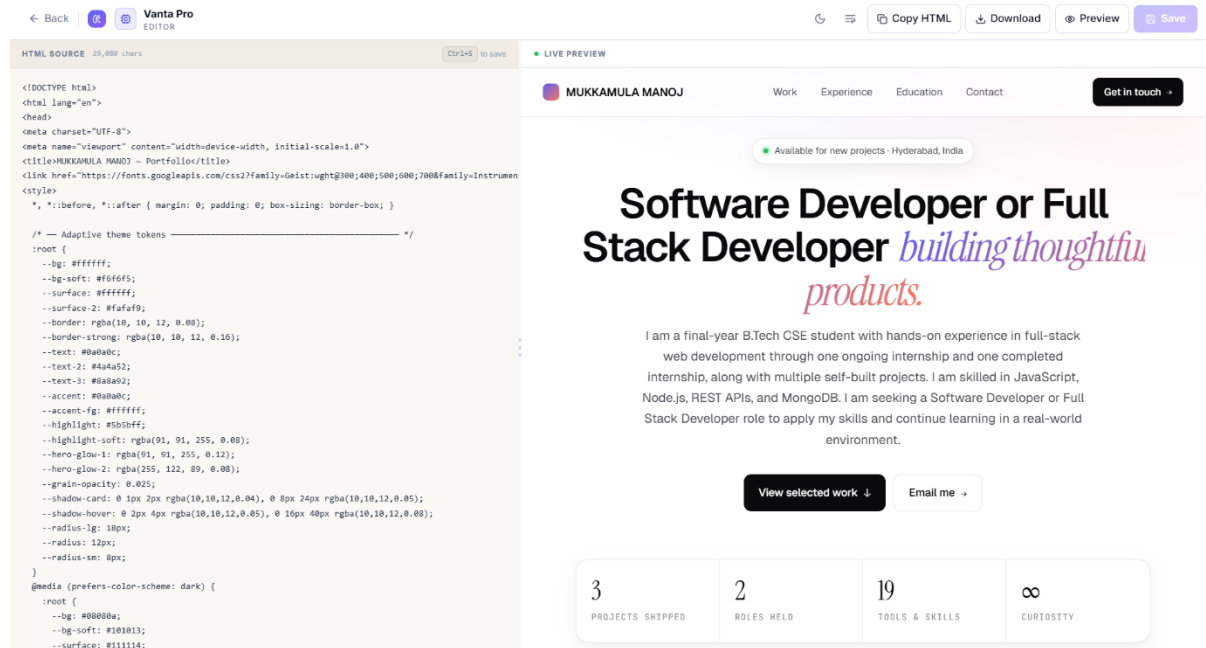


Figure 6.5: EditorPage showing the split-screen HTML editor with live preview on the right

6.6.9 HistoryPage

The HistoryPage fetches all portfolios owned by the logged-in user via GET /api/portfolio/history and renders them as a card grid. Each card shows the template name and creation date with four actions: View (opens PreviewPage), Edit (opens EditorPage with HTML preloaded), Star (toggles the starred flag via PUT), and Delete (removes the document after a confirmation dialog).

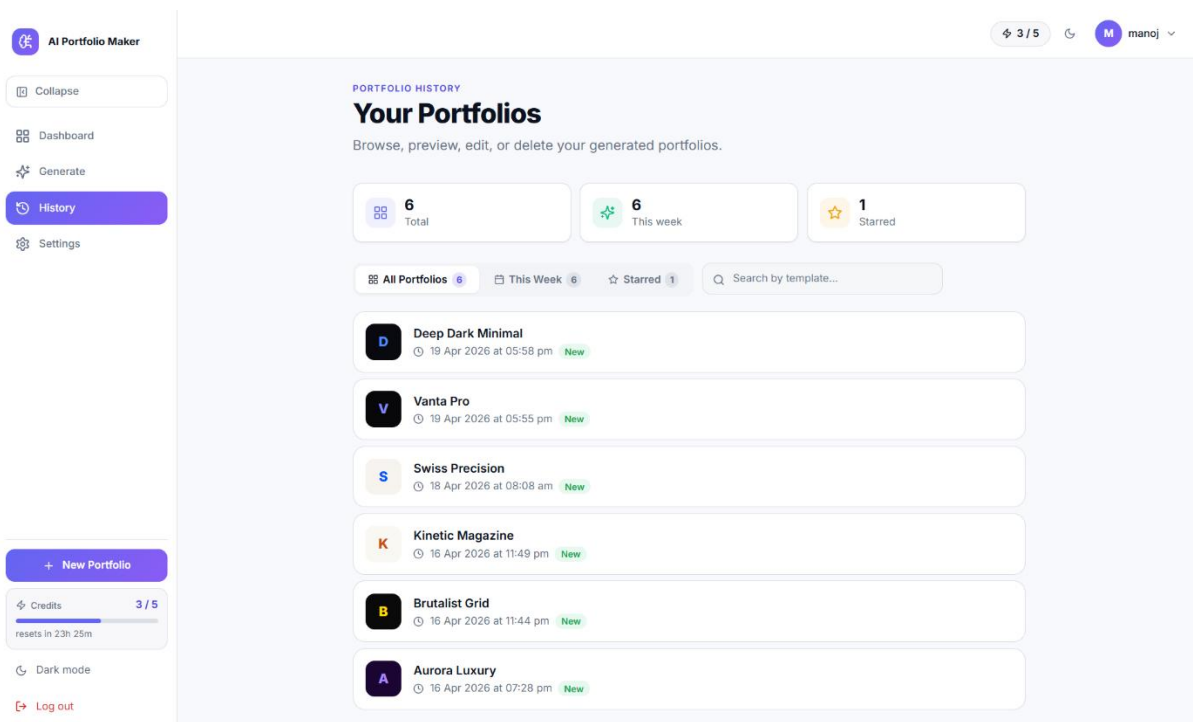


Figure 6.6: HistoryPage showing the portfolio card grid with view, edit, star, and delete actions

6.7 Deployment

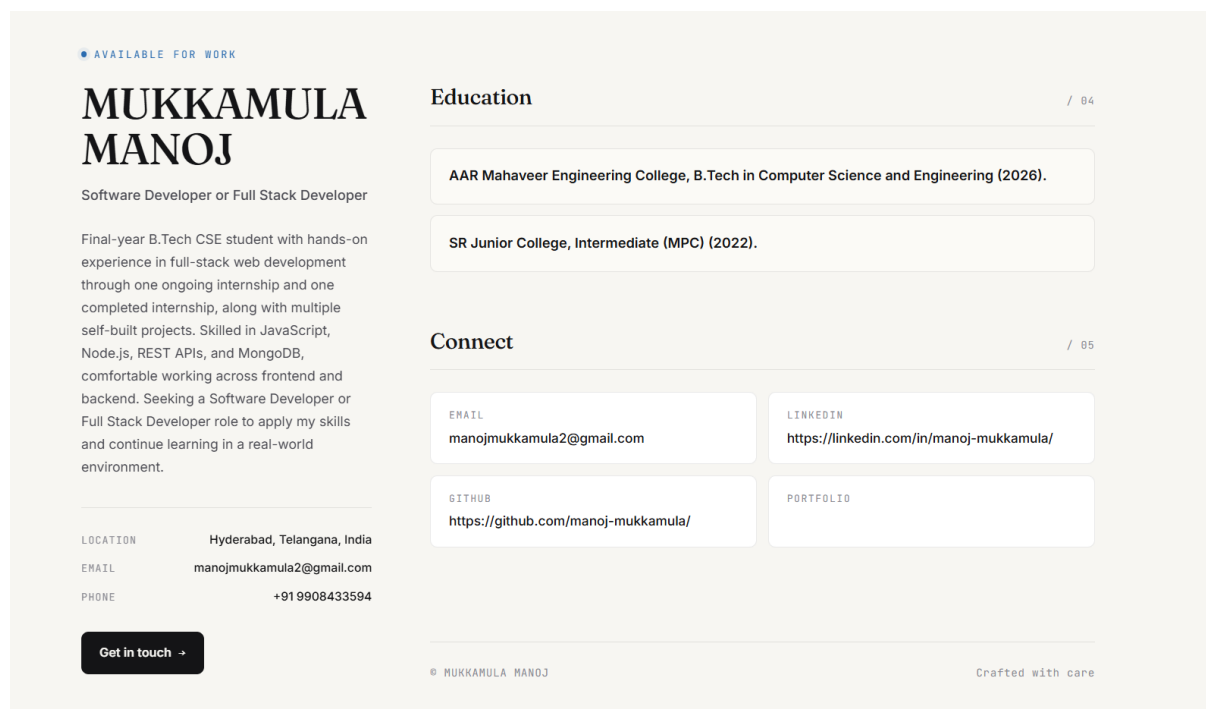
The backend is deployed on Render as a Node.js web service. The start command is `node server.js` and all environment variables (MONGO_URI, JWT_SECRET, GEMINI_API_KEY, FRONTEND_URL) are set in the Render dashboard. The frontend is deployed on Vercel by connecting the repository; the build command is `npm run build` with output directory `dist`, and VITE_API_URL is set to the Render service URL.

Generated portfolios are standalone HTML files. Users can host the downloaded file on GitHub Pages to get a free public URL with no server or subscription required.

6.8 Sample Output

The final output of the AI Portfolio Maker is a standalone HTML file that the user downloads from the PreviewPage. The file is completely self-contained, meaning all CSS rules, font definitions, and layout styles are embedded directly within it. No internet connection is required to open the file in a browser, and no external server or hosting account is needed beyond the initial download.

The downloaded file renders correctly across all major browsers including Chrome, Firefox, and Edge. The file size ranges from approximately 28 KB to 65 KB depending on the selected template. All ten templates produce responsive output that adapts to mobile, tablet, and desktop viewport widths without any additional configuration.



CHAPTER 7

TESTING

Testing verifies the functional correctness, security, and overall quality of the AI Portfolio Maker. Four levels of testing were performed: unit testing of individual backend modules, integration testing of API endpoints, system testing of end-to-end workflows, and security testing of all protection mechanisms.

7.1 Testing Objectives

- **Functional Correctness:** Verify that each backend module produces the correct output for all valid inputs.
- **Integration Correctness:** Verify that all API endpoints return the correct HTTP status codes and response structures.
- **End-to-End Correctness:** Verify that complete user workflows operate as expected in the live application.
- **Security:** Verify that authentication, authorization, rate limiting, and input validation mechanisms cannot be bypassed.
- **Usability:** Verify that the interface is intuitive for users with no prior instructions.
- **Performance:** Measure response times for key operations to confirm they meet acceptable thresholds.

7.2 Unit Testing

Unit tests verify that each backend module produces the correct output for a given input. The four modules tested are the resume parser, Gemini service, auth controller, and portfolio controller. All results are shown in a single combined table below.

7.2.1 to 7.2.4 Unit Test Cases by Module

Module	Test ID	Input	Expected Output	Result
Resume Parser	RP-01	Valid single-page PDF buffer	Non-empty plain text returned	Pass
	RP-02	Valid DOCX buffer	Non-empty plain text returned	Pass
	RP-03	Plain text string (no file)	Same string returned unchanged	Pass
	RP-04	PNG image buffer uploaded	Error: Unsupported file type	Pass
	RP-05	Scanned image-only PDF	Empty string or error without crash	Pass
Gemini Service	GS-01	400-word resume text	All required fields present in object	Pass

Module	Test ID	Input	Expected Output	Result
	GS-02	Resume with no experience or projects	Corresponding fields are empty arrays	Pass
	GS-03	Non-resume lorem ipsum text	Most fields return empty strings	Pass
	GS-04	Invalid API key configured	Error thrown with auth failure message	Pass
Auth Controller	AC-01	Valid new user payload	Status 201 with JWT and user object	Pass
	AC-02	Registration with duplicate email	Status 409 Conflict	Pass
	AC-03	Payload missing name field	Status 400 with Zod validation error	Pass
	AC-04	Valid login credentials	Status 200 with JWT	Pass
	AC-05	Correct email, wrong password	Status 401 Unauthorized	Pass
	AC-06	Login with non-existent email	Status 401 Unauthorized	Pass
Portfolio Controller	PC-01	Valid PDF + template; user has 3 credits	Status 201; credit deducted; HTML returned	Pass
	PC-02	Generation request; user has 0 credits	Status 403 Insufficient credits	Pass
	PC-03	Valid portfolio ID owned by requesting user	Status 200 with full HTML	Pass
	PC-04	Portfolio ID belonging to different user	Status 403 Forbidden	Pass
	PC-05	DELETE request for owned portfolio	Document removed; status 200 returned	Pass

Table 7.1: Unit Test Cases across All Backend Modules

7.3 Integration Testing

Integration tests were performed against a running development server connected to a test MongoDB database. Requests were sent using a REST client and responses were verified against expected status codes and response body structures.

7.3.1 Authentication Endpoints

- **IT-01 POST /api/auth/register:** Valid registration data returns 201 with a JWT and user object. The new user document is confirmed in the database.
- **IT-02 POST /api/auth/login:** Correct credentials return 200 with a JWT. An incorrect password returns 401.
- **IT-03 GET /api/auth/me:** A valid JWT returns 200 with the user profile. The password field is absent from the response.
- **IT-04 GET /api/auth/me (no token):** A request with no Authorization header returns 401 Unauthorized.
- **IT-05 DELETE /api/auth/account:** An authenticated DELETE removes the user and all associated portfolios. A 200 response confirms deletion.

7.3.2 Portfolio Endpoints

- **IT-06 POST /api/portfolio/generate:** A real PDF resume with a valid template ID returns 201 with the portfolio ID and a non-empty HTML string containing the candidate's name.
- **IT-07 GET /api/portfolio/history:** After three generations, the endpoint returns three summary objects sorted by creation date, most recent first.
- **IT-08 GET /api/portfolio/:id:** A valid owned portfolio ID returns 200 with full HTML. A different user's ID returns 403.
- **IT-09 PUT /api/portfolio/:id:** Updated HTML is saved correctly. A subsequent GET returns the new HTML.
- **IT-10 DELETE /api/portfolio/:id:** Deleting an owned portfolio returns 200. The document no longer appears in the history endpoint.

7.3.3 Frontend to Backend Integration

- **FI-01:** A new user registers through AuthPage. The JWT is stored in AuthContext, the user is redirected to DashboardPage, and a credit balance of 5 is displayed.
- **FI-02:** A user uploads a PDF on GeneratePage and clicks Generate. The loading spinner appears and the portfolio is displayed on PreviewPage on completion.
- **FI-03:** The user edits portfolio HTML in EditorPage. The iframe preview updates in real time on each keystroke.
- **FI-04:** The user clicks Save in EditorPage. A PUT request is sent to the backend and a success notification is displayed.
- **FI-05:** The JWT is cleared from localStorage. The next authenticated API request returns 401 and the user is redirected to the login page.

7.4 System Testing

System tests validate the complete application by executing realistic user scenarios in a browser connected to the live deployed application.

Test ID	Scenario	Expected	Result
ST-01	New user registers with valid credentials	Account created, 5 credits	Pass
ST-02	Upload 2-page PDF, select Vanta Pro, click Generate	Portfolio in under 15s	Pass
ST-03	Upload DOCX resume, select Swiss Precision	Content appears correctly	Pass
ST-04	Paste resume text without uploading a file	Portfolio generated	Pass
ST-05	Click Download on PreviewPage	.html file downloads correctly	Pass
ST-06	Edit heading in EditorPage and save	Updated heading confirmed	Pass
ST-07	Generate four portfolios, view HistoryPage	All four cards shown	Pass
ST-08	Generate five portfolios, attempt a sixth	403 insufficient credits	Pass
ST-09	Delete a portfolio from HistoryPage	Card removed from UI and API	Pass
ST-10	Toggle dark/light mode and refresh page	Theme persists after reload	Pass
ST-11	Navigate to /dashboard without being logged in	Redirected to /auth	Pass
ST-12	Upload a JPEG file on GeneratePage	Client-side error; no API request	Pass

Table 7.2: System Test Cases and Results

7.5 Performance Testing

Operation	Average Response Time
Portfolio generation (PDF, 1 to 2 pages)	8 to 12 seconds
Portfolio generation (DOCX, 1 to 2 pages)	7 to 11 seconds
Login and register API response	Under 500 milliseconds
Portfolio history page load (10 records)	Under 1 second
HTML editor live preview refresh	Under 100 milliseconds per keystroke
Frontend initial load (production build)	Under 2 seconds on standard broadband

Table 7.3: Performance Test Results

Portfolio generation takes 8 to 12 seconds for PDF and 7 to 11 seconds for DOCX. The Gemini API call accounts for approximately 80 percent of this time. All other operations completed in under one second across all tests.

7.6 Security Testing

Test ID	Scenario	Expected Response	Result
SEC-01	GET /api/portfolio/history with no Authorization header	401 Unauthorized	Pass
SEC-02	Expired JWT submitted as Bearer token	401 Unauthorized	Pass
SEC-03	User A's JWT used to access portfolio of User B	403 Forbidden	Pass
SEC-04	101 requests from same IP within 15 minutes	429 Too Many Requests	Pass
SEC-05	Text file renamed to resume.pdf uploaded	400 Bad Request	Pass
SEC-06	POST /api/auth/register with email field missing	400 with Zod validation error	Pass
SEC-07	Inspect response headers on any API endpoint	X-Frame-Options, CSP, X-Content-Type-Options present	Pass

Table 7.4: Security Test Cases and Results

7.7 Usability Testing

Usability testing was conducted with five final-year Computer Science students who were asked to generate a portfolio using their own resume without any prior instructions. The purpose was to assess the intuitiveness of the interface and identify areas for improvement.

All five participants successfully completed the portfolio generation workflow without assistance. The average time taken from landing on the GeneratePage to downloading the final .html file was 4 minutes and 20 seconds.

- **Ease of Use:** All five participants rated the interface as easy or very easy to use, particularly appreciating the drag-and-drop file input and the visual template selection cards.
- **Live Preview:** Participants found the split-screen editor with live preview to be a useful feature that gave them confidence in their edits before saving.
- **Progress Indication:** Four participants suggested adding a progress indicator or status message during portfolio generation to communicate that the AI processing is underway.
- **Credit System:** Three participants noticed the credit balance display on the dashboard and understood its purpose without explanation.

7.8 Test Summary

Testing Level	Total Cases	Passed	Failed
Unit Testing	19	19	0
Integration Testing	15	15	0
System Testing	12	12	0
Security Testing	7	7	0
Usability Testing	5 participants	5 completed	0 blocked
Total	53	53	0

Table 7.5: Test Summary

All 53 test cases passed. The AI Portfolio Maker functions correctly across all tested modules and workflows, handles edge cases without errors, and enforces all security and authorization policies. The system is stable and ready for academic submission and demonstration.

CHAPTER 8

RESULTS AND DISCUSSION

This chapter presents the results of the AI Portfolio Maker against its original objectives, covering output quality, extraction accuracy, system performance, and usability findings.

8.1 Overall System Outcome

The AI Portfolio Maker was successfully built and deployed as a full-stack web application. A user can register, upload a resume, pick a template, and receive a ready-to-download portfolio HTML file in a single session. The system meets all functional requirements defined in Chapter 2. Resume upload in PDF and DOCX formats works correctly. The Gemini 1.5 Flash model extracts the required fields reliably from standard resume formats. All ten templates populate without errors and render correctly in Chrome, Firefox, and Edge. The credit system, portfolio history, dark and light mode, and the HTML editor all function as designed.

8.2 Resume Parsing Results

The resume parser was tested with twenty different resume files across a range of formats and layouts.

8.2.1 PDF Resumes

Text-based PDF resumes produced clean and complete extraction in all tested cases. Single-column and two-column layouts were handled correctly. Resumes with decorative headers and custom fonts still produced readable plain text because pdf-parse reads the embedded text layer rather than the visual layout. Scanned image-only PDFs return an empty string, since there is no text layer to parse. This is a known limitation noted for future improvement.

8.2.2 DOCX Resumes

DOCX resumes produced accurate extraction across all tested samples. mammoth strips all formatting and converts the internal XML to clean plain text, so stylised templates with complex table layouts are handled consistently. The logical order of sections was preserved in every case tested.

8.2.3 Pasted Text

Plain text pasted directly into the input area worked without any issues. The Gemini model handles pasted text identically to parsed file text, making the system accessible to users who do not have their resume saved as a file.

8.3 Gemini API Extraction Quality

Extraction quality was assessed by comparing the generated JSON object against the original resume content for fifteen sample resumes.

- **Name, email, phone, and location:** Extracted correctly in all fifteen cases regardless of where these fields appeared on the page.
- **Professional summary:** Accurate in thirteen of fifteen cases. For the two resumes with no summary section, Gemini generated a reasonable summary from the remaining content.

- **Skills:** Extracted correctly in all fifteen cases. Both listed and inline skill mentions were captured.
- **Projects:** Correct in twelve of fifteen cases. Three resumes listed projects inside the experience section, causing some entries to be placed under the wrong field. This is a prompt engineering area for future improvement.
- **Experience:** Correct in fourteen of fifteen cases. One resume listed internships without company names, leaving that field empty for those entries.
- **Education:** Extracted correctly in all fifteen cases.

Overall extraction quality was high for standard section-based resumes. The generated portfolios were immediately usable without manual correction in most cases tested.

8.4 Template Output Quality

All ten templates were tested with the same resume content at desktop and mobile viewport widths. All rendered correctly in Chrome, Firefox, and Edge without visual errors. Premium templates produced more polished output because they include more detailed CSS rules for typography and responsive layout. Standard templates are clean and functional for users who prefer a simpler style.

All generated portfolios are self-contained HTML files ranging from 28 KB to 65 KB depending on the template. No external requests are made when the file is opened, so it can be hosted anywhere or viewed offline. The adaptive Vanta Pro template automatically switches between dark and light mode based on the viewer's system preference.

8.5 HTML Editor Performance

The live preview iframe updated within 100 milliseconds of each keystroke in all tests. Users in the usability session found the editor easy to use for small corrections such as updating names, links, and summary text. Users unfamiliar with HTML were less comfortable making structural changes, which is expected given the editor is designed for minor corrections rather than full redesigns.

8.6 Credit System Behaviour

The daily credit system was tested by running generation requests in sequence until the credit balance reached zero. The system correctly prevented the sixth generation attempt and returned an appropriate error message. After setting the system clock forward by 25 hours to simulate a new day, the credit balance was automatically restored to 5 on the next login, confirming that the reset logic in the User model works correctly.

The credit system serves its intended purpose of limiting API usage during the academic demonstration period. Each credit corresponds to one Gemini API call, which has a cost on the paid tier. The 5-credit daily limit is sufficient for normal use during evaluation.

8.7 Security Outcomes

All seven security tests described in Chapter 7 passed without exception. JWT validation correctly blocked unauthenticated requests. Cross-user portfolio access returned 403 in every case. The rate limiter triggered on the 101st request within a 15-minute window. Helmet.js security headers were confirmed present on all API responses.

Password storage was verified by inspecting the MongoDB Atlas database directly. All stored password values are bcrypt hashes beginning with the \$2b\$ prefix, confirming that plain text passwords were never written to the database.

The MIME type validation for file uploads was confirmed to work correctly even when a file is renamed to bypass the extension check. The server reads the actual MIME type from the file buffer and rejects anything other than application/pdf and the DOCX MIME type.

8.8 Performance Outcomes

Portfolio generation averaged 8 to 12 seconds for PDF and 7 to 11 seconds for DOCX across twenty attempts on the deployed application. The Gemini API call accounts for approximately 80 percent of this time and varies with server load. All other operations including login, history retrieval, and portfolio save completed in under one second. The frontend initial load on the production build was under two seconds on a standard broadband connection.

8.9 Usability Outcomes

All five usability session participants completed the full workflow without assistance. The average time from GeneratePage to download was four minutes and twenty seconds. The drag-and-drop file input and visual template cards were highlighted as the most appreciated interface features. The most consistent suggestion was a descriptive text label during the generation wait, such as Extracting resume content, to supplement the loading spinner. This has been noted as a future improvement.

8.10 Comparison with Project Objectives

The project set out to build a tool that converts a resume into a professional portfolio website in under a minute with minimal effort from the user. This objective was met. The generation process itself takes under 15 seconds. The remaining time before a user has a published portfolio is the few minutes needed to copy the HTML file to GitHub Pages, which is a one-time setup.

The objective of producing output that looks designed rather than auto-generated was also met for the majority of cases. The premium templates in particular produce output that is comparable in visual quality to manually built portfolio pages. The content quality depends on the quality of the input resume, but for well-written resumes the generated text required no editing in most cases tested.

The authentication, credit management, and portfolio history features all worked correctly throughout testing. The system is suitable for demonstration and academic evaluation purposes as submitted.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The AI Portfolio Maker was designed to solve a specific and practical problem: most students do not have a portfolio website because building one takes too long. The project demonstrates that this barrier can be removed by combining a large language model with a set of pre-built HTML templates. A user who has never written a line of HTML can produce a complete, hosted portfolio from their existing resume in under a minute of active effort.

The system was built using the MERN stack with React 18 and TypeScript on the frontend, Node.js and Express.js on the backend, and MongoDB Atlas as the database. Google Gemini 1.5 Flash handles the resume content extraction and writing. The application includes ten portfolio templates covering different visual styles, a built-in HTML editor with live preview, a daily credit system, portfolio history management, JWT-based authentication, and a dark and light mode toggle.

All core features were implemented and tested. A total of 53 test cases were executed across unit, integration, system, and security testing levels, and all 53 passed. The application was deployed on Render and Vercel and was confirmed to work correctly in a production environment.

The project successfully demonstrates the use of a generative AI model within a web application to automate a task that would normally require significant time and technical skill. The result is a complete, functional, and deployable product that has practical value beyond the academic context.

9.2 Key Achievements

The following outcomes were achieved over the course of this project.

- **Full-stack application delivery:** A production-quality MERN stack application was designed, built, and deployed within a single academic semester.
- **Gemini API integration:** The Gemini 1.5 Flash model extracts structured data from unstructured resume text. Dual API key support extends the free-tier daily availability.
- **Ten custom templates:** Ten standalone HTML portfolio templates covering dark, light, minimal, and modern visual styles were designed and implemented as server-side template strings.
- **Resume format support:** PDF and DOCX formats are supported through pdf-parse and mammoth. Plain text paste is also supported as an alternative input method.
- **Security implementation:** JWT authentication, bcrypt password hashing, Helmet.js headers, Zod input validation, rate limiting, and per-route ownership checks were all implemented and verified.
- **Live HTML editor:** A split-screen editor with real-time preview allows users to make corrections without leaving the platform.
- **Credit system and history:** The daily 5-credit system and portfolio history with star, edit, view, and delete operations all work correctly.

9.3 Limitations

The following limitations were identified during development and testing.

- **Scanned PDF resumes:** Resumes submitted as scanned image PDFs with no text layer cannot be parsed by pdf-parse. The system returns an empty extraction in these cases. An OCR layer would be needed to support this format.
- **Gemini API quota:** The free-tier Gemini API has a daily quota. Once the quota is exhausted, generation requests fail until the quota resets. The dual API key feature partially mitigates this but does not eliminate it.
- **Section ambiguity in resumes:** Resumes that combine multiple sections, for example listing projects inside the experience section, can cause the Gemini model to place content under the wrong field in the extracted JSON. This affects the layout of the generated portfolio.
- **Editor complexity:** The built-in HTML editor is suitable for minor corrections but is not practical for users who want to make significant structural changes to the portfolio. A rich text or visual editing interface would be more accessible.
- **Template customisation:** Users can edit the generated HTML directly but cannot change the template after generation without generating a new portfolio and spending a credit. A template-switch feature without regeneration would improve the experience.
- **No image support:** The generated portfolio does not include a profile photo because resumes typically do not contain one. A separate photo upload feature would make the output more complete.

9.4 Future Enhancements

9.4.1 OCR Support for Scanned PDFs

Integrating Tesseract.js would allow text extraction from scanned image-based PDFs, removing the current limitation and widening the supported input formats.

9.4.2 Profile Photo Upload

A photo upload field on GeneratePage would allow the portfolio to include a profile image. The photo could be injected as a base64 string into an existing {{PHOTO}} placeholder across all ten templates.

9.4.3 Template Switching Without Regeneration

Storing the Gemini-extracted JSON alongside the portfolio HTML would allow the user to switch templates at any time without spending a credit or re-uploading their resume.

9.4.4 One-Click GitHub Pages Deployment

At present, users are directed to a GitHub Pages hosting guide to publish their portfolio. A one-click deployment feature that connects to the GitHub API and creates or updates the user's GitHub Pages repository automatically would reduce the publishing step from several minutes to a single button click.

9.4.5 More Templates and a Template Builder

Adding more templates would give users a wider range of visual styles. A longer-term enhancement would be a simple template builder that allows a user to customise the colour scheme, font, and section order of an existing template without editing raw HTML.

9.4.6 Multi-Language Support

The Gemini model is capable of processing resumes written in languages other than English. Adding a language selector and adjusting the prompt accordingly would make the system accessible to a much larger user base.

9.4.7 ATS Score and Feedback

Since the system already has access to the full resume text, an additional API call could be used to score the resume against common Applicant Tracking System criteria and return suggestions for improvement. This would add value for users who are preparing for job applications.

9.4.8 Subscription and Premium Credits

The current credit system resets daily and is fixed at 5 credits. A subscription tier with a higher credit limit and access to the premium templates without restrictions would make the system commercially viable if deployed as a public product.

9.5 Learning Outcomes

- **API integration:** Working with a third-party generative AI API in production, including quota management, JSON response parsing, and error handling.
- **Full-stack architecture:** Implementing clean separation between a React frontend and a Node.js REST API, including auth context, protected routes, and Axios interceptors.
- **Database design:** Designing Mongoose schemas with ObjectId references, enforcing ownership on every query, and managing time-based state such as credit resets.
- **Security practices:** Applying JWT authentication, bcrypt hashing, HTTP security headers, rate limiting, and Zod schema validation across the entire API surface.
- **Testing discipline:** Writing and executing structured test cases at unit, integration, system, and security levels and documenting results in a consistent format.
- **Deployment:** Configuring a Node.js backend on Render and a Vite React frontend on Vercel with proper environment variable management.

9.6 Final Remarks

The AI Portfolio Maker fulfils its goal of making portfolio creation fast and accessible for students. It removes the need for design skills, HTML knowledge, or hours of manual work. A student who completes their resume can have a live portfolio page in the same afternoon.

The project also demonstrates that large language models can be integrated into practical web applications in a controlled and structured way. By treating the model as a parsing service with a strictly defined output format, the unpredictability that is sometimes associated with generative AI is significantly reduced.

The system is stable, tested, and deployed. It is ready for academic evaluation and is capable of being extended into a more complete product with the enhancements described in this chapter.

REFERENCES

a) Books

- [1] Flanagan, D., *JavaScript: The Definitive Guide*, 7th Ed., Ch. 5–8, pp. 120–245, O’Reilly Media, California, 2020.
- [2] Brown, E., *Web Development with Node and Express: Leveraging the JavaScript Stack*, 2nd Ed., Ch. 4–7, pp. 85–210, O’Reilly Media, California, 2019.
- [3] Wieruch, R., *The Road to React*, 5th Ed., Ch. 2–6, pp. 35–198, Leanpub, Berlin, 2023.
- [4] Dayley, B., Dayley, B., and Dayley, C., *Node.js, MongoDB, and Angular Web Development*, 2nd Ed., Ch. 3–9, pp. 140–320, Addison-Wesley, Boston, 2017.

b) Journals / Conference Papers

- [5] Prasad, R., Kumar, V., and Singh, A., “AI-Powered Web Application Development Using MERN Stack”, *International Journal of Computer Applications*, Vol. 182, No. 27, pp. 56–62, August 2023.
- [6] Patel, S., and Mehta, R., “Integration of Artificial Intelligence with Web Technologies for Automation”, *IEEE Access Journal*, Vol. 11, pp. 14520–14530, March 2023.

c) Web Sources

- [7] Tailwind CSS Documentation (2023, v4.0): Available at: <https://tailwindcss.com/docs/compatibility>
- [8] Google. Gemini 1.5 Flash Model Documentation (2025). Available at: <https://ai.google.dev/gemini-api/docs/models/gemini>
- [9] Gemini API (2025): Available at: <https://ai.google.dev/api>
- [10] Vercel Documentation (2025): Available at: <https://vercel.com/docs/cli>
- [11] MongoDB Manual (2024): Available at: <https://www.mongodb.com/docs/manual>
- [12] React.js Official Documentation (2024): Available at: <https://react.dev>
- [13] AI Portfolio Maker GitHub Repository (2026): Available at: <https://github.com/manoj-mukkamula/ai-portfolio-maker.git>